

COMP1786

Mobile Application Design and Development Report

Name	Student ID
Kaung Satt Hein	001474412

Table of Contents

SECTION 1 - BRIEF STATEMENT OF FEATURES.....	3
SECTION 2 - SCREEN SHOTS/DESIGN.....	7
Feature (A).....	7
Feature (B).....	11
Feature (C).....	14
Feature (D).....	18
Function (E).....	20
Function (F).....	22
Function (G).....	27
Figure 21: Advance Feature Screen.....	27
SECTION 3 - REFLECTION.....	28
SECTION 4 - EVALUATION.....	30
Introduction.....	30
Human-Computer Interaction (HCI).....	30
Security.....	31
Screen Size Compatibility.....	31
Deployment Readiness.....	32
SECTION 5 - CODE.....	34

SECTION 1 - BRIEF STATEMENT OF FEATURES

Feature	Status	Comment
Functionality A	☒ Fully completed ☐ Partially completed ☐ Having bugs/Not working ☐ Not implemented	I was able to implement successfully the hiking data entry form, with the help of which the user will be able to input all the obligatory and optional fields. The field names like the hike name, address, date, the parking availability and length and the level of difficulty are mandatory and are checked. In case the user fails to fill in any of the error message will be given as a mandatory information. come up. I introduced another three as well. creative areas: a rating scale (1-5 stars) to appraise trek, an image addition option (user can) and take a picture, or select one in the gallery, and weather condition (radio buttons sunny, overcast, rainy, etc.). All input fields work and the information is stored in the local, and it is done correctly. database successfully.
Functionality B	☒ Fully completed ☐ Partially completed ☐ Having bugs/Not working ☐ Not implemented	Full CRUD was applied using SQLite. The users can store the data of Hikes, and see all the additions in a list, modify or delete a Hike and reset.

		the complete database. Even when the application is closed, real-time updates of information are still on. There are confirmation dialogs that have been introduced into the case. of deleting and resetting among other operations.
Functionality C	☒ Fully completed ☐ Partially completed ☐ Having bugs/Not working ☐ Not implemented	Users are able to choose a hike and make several. observations that are stored in SQLite. Each observation has fields of description and time that are required. (filled in with current date/time) and an optional comment. The users are able to view, edit and delete. observations for each hike. All functions work immediately and automatically update.
Functionality D	☒ Fully completed ☐ Partially completed ☐ Having bugs/Not working ☐ Not implemented	Introduced a search bar where one could search in advance search hikes. The search is kept up to date in the form of users type, listing all the matches with the entered letters. Users have the option of tapping a result to read fully. hike details. Search works effectively with the database and presents correct results.
Functionality E	☒ Fully completed ☐ Partially completed ☐ Having bugs/Not working	Enacted full hike entry feature (all).

	☐ Not implemented	required optional validation confirmation fields. screen) using React Native. UI mirrors the native Precisely, Android version, with suitable. controls: Radio, DatePicker, Radion, TextField. buttons, etc. Works completely on Android and. iOS via React Native.
Functionality F	☒ Fully completed ☐ Partially completed ☐ Having bugs/Not working ☐ Not implemented	Total persistence through React Native with. sqlite. Every increase and observation is registered. local in relational SQLite tables. Supports insert, There are update, delete, and delete-all operations with. appropriate foreign key relationships. Data survives app restarts and reboots.
Functionality G	☐ Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented	There are two strong improvements that are used in. Android (Kotlin): Photo per hike Photo attachments Photo per hike. (take photo with camera), presented in scrollable gallery, stayed intact completely. • Real location capture - Auto recording. when the user is at a given location, the GPS of that location. adding/editing a hike (with permission handling).

		In the, location has been saved as latitude and longitude. database and shown in the hike details (e.g., “Hike started at: 51.5074° N, 0.1278° W”). Users can also, update or look at the location manually in case necessary.
--	--	--

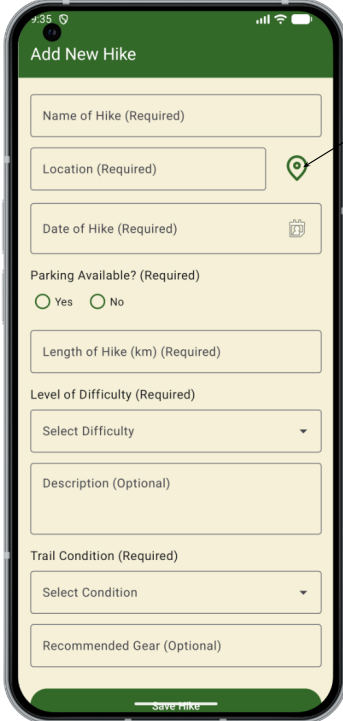
Screen Recording :

https://drive.google.com/drive/u/0/folders/105Pqgj9wHY_2eYWYDrVQE_gy1UyV9IyH

SECTION 2 - SCREEN SHOTS/DESIGN

Feature (A)

The app actively checks that all fields marked "(Required)" are filled before saving, preventing the submission of incomplete forms.v



Tapping the map pin icon immediately detects and fills the input field with the user's current location, eliminating the need for manual typing.

Add New Hike

Name of Hike (Required)

Location (Required)

Date of Hike (Required)

Parking Available? (Required)

☐ Yes ☐ No

Length of Hike (km) (Required)

Level of Difficulty (Required)

Select Difficulty

Description (Optional)

Trail Condition (Required)

Select Condition

Recommended Gear (Optional)

Save Hike

Figure 1: Add New Hike

When required fields are left empty, red exclamation mark icons appear directly inside the text input boxes (e.g., "Name of Hike"), providing immediate visual feedback that attention is needed.

The figure displays three sequential screenshots of a mobile application's 'Add New Hike' form, illustrating the visual feedback provided for required fields.

Screenshot 1 (Left): The form is shown with several required fields (Name of Hike, Location, Date of Hike, Parking Available, Length of Hike, Level of Difficulty, Trail Condition) and optional fields (Description, Recommended Gear). A red exclamation mark icon is visible in the 'Name of Hike' field, indicating it is required and currently empty.

Screenshot 2 (Middle): The user has entered 'Mount Zwegabin' in the 'Name of Hike' field. The red exclamation mark icon is still present in the 'Name of Hike' field, indicating it is still required.

Screenshot 3 (Right): The user has entered 'Yangon, Myanmar (Burma)' in the 'Location' field. The red exclamation mark icon is still present in the 'Name of Hike' field, indicating it is still required.

The form includes the following fields:

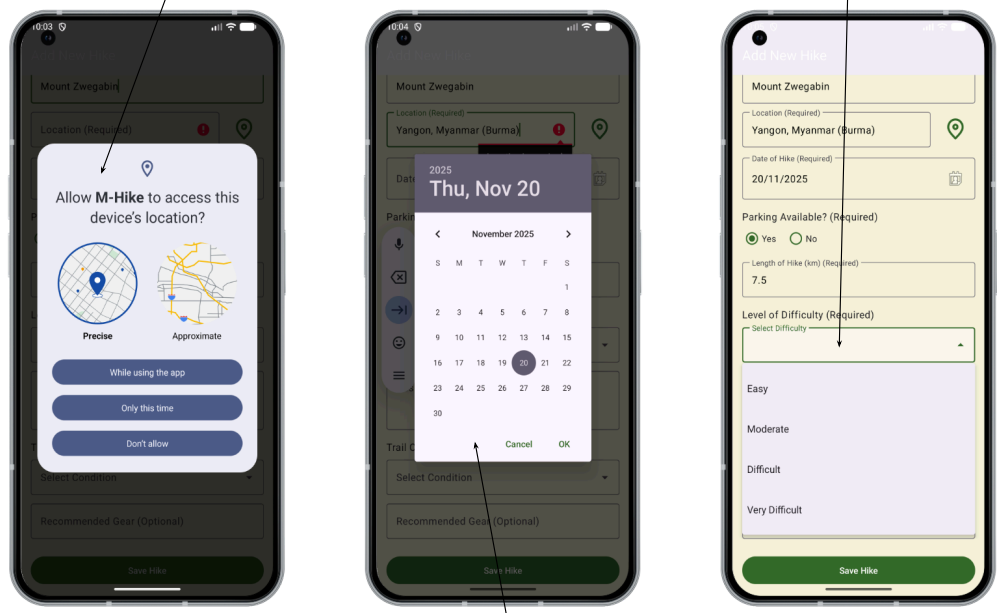
- Name of Hike (Required)
- Location (Required)
- Date of Hike (Required)
- Parking Available? (Required)
 - ☐ Yes
 - ☐ No
- Length of Hike (km) (Required)
- Level of Difficulty (Required)
 - Select Difficulty
- Description (Optional)
- Trail Condition (Required)
 - Select Condition
- Recommended Gear (Optional)

The 'Save Hike' button is visible at the bottom of the form.

Figure 2: Add New Hike

Before accessing GPS data, the app triggers a standard system dialog requesting location permissions. This ensures user consent and security before enabling the auto-location feature.

The "Level of Difficulty" menu expands when tapped, revealing the pre-defined options (Easy, Moderate, etc.). This "Exposed Dropdown" style keeps the UI clean while preventing invalid data entry.



Tapping the date field opens a native calendar dialog overlay. This provides a familiar, touch-friendly interface that ensures the date format is always valid and consistent.

Figure 3: Add New Hike

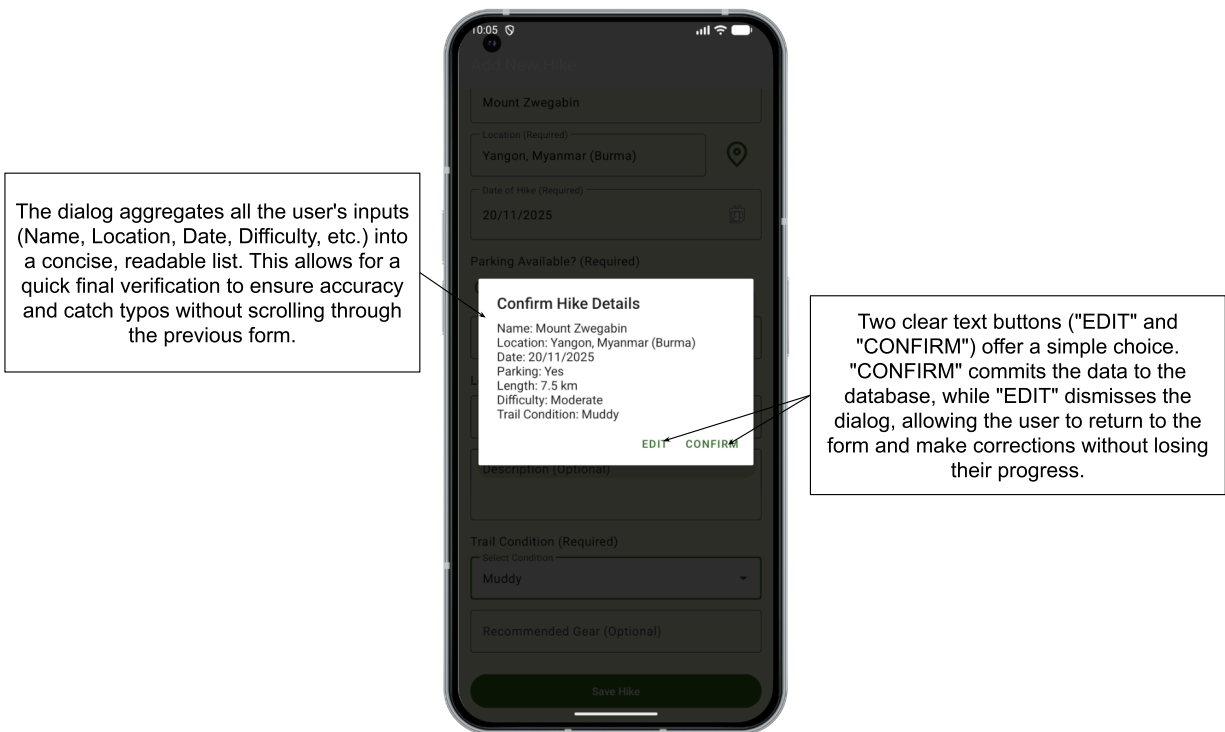


Figure 4: Confirm Hike Details

Feature (B)

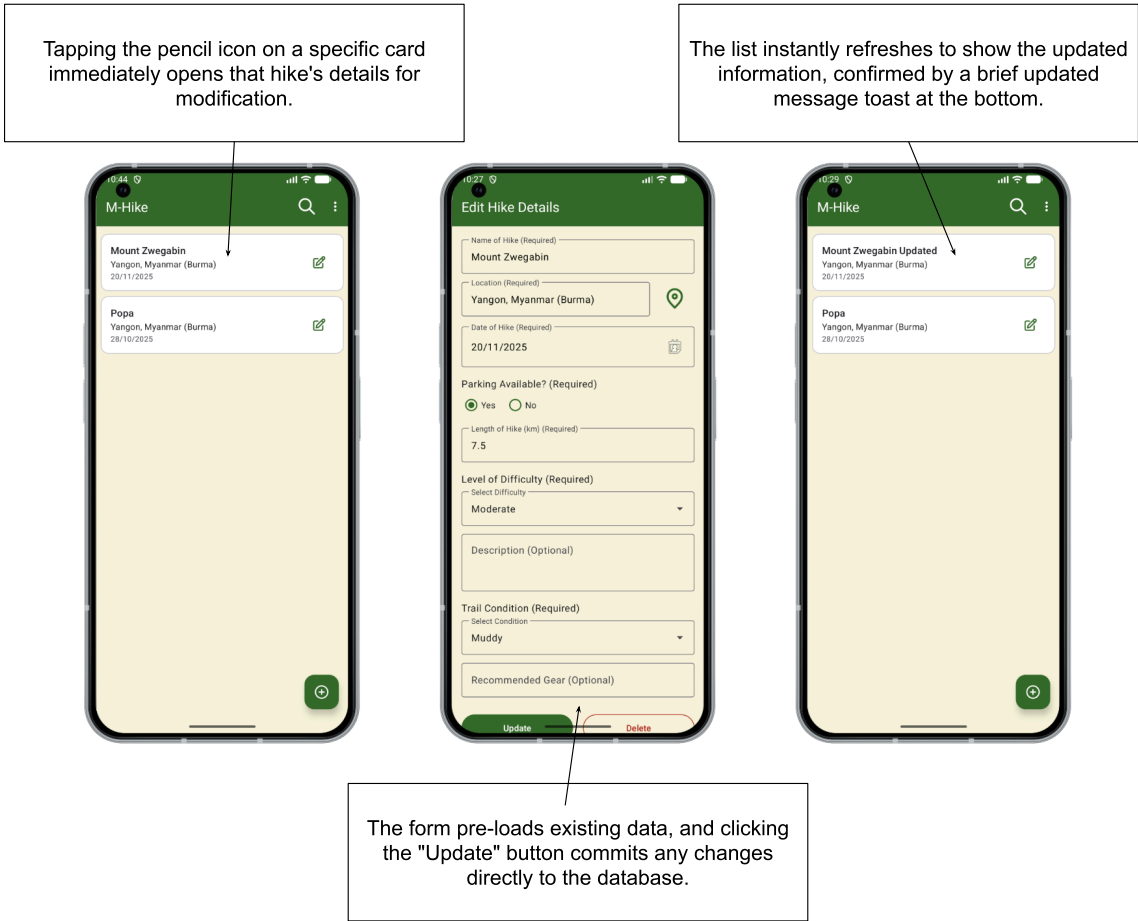


Figure 5: Home Screen

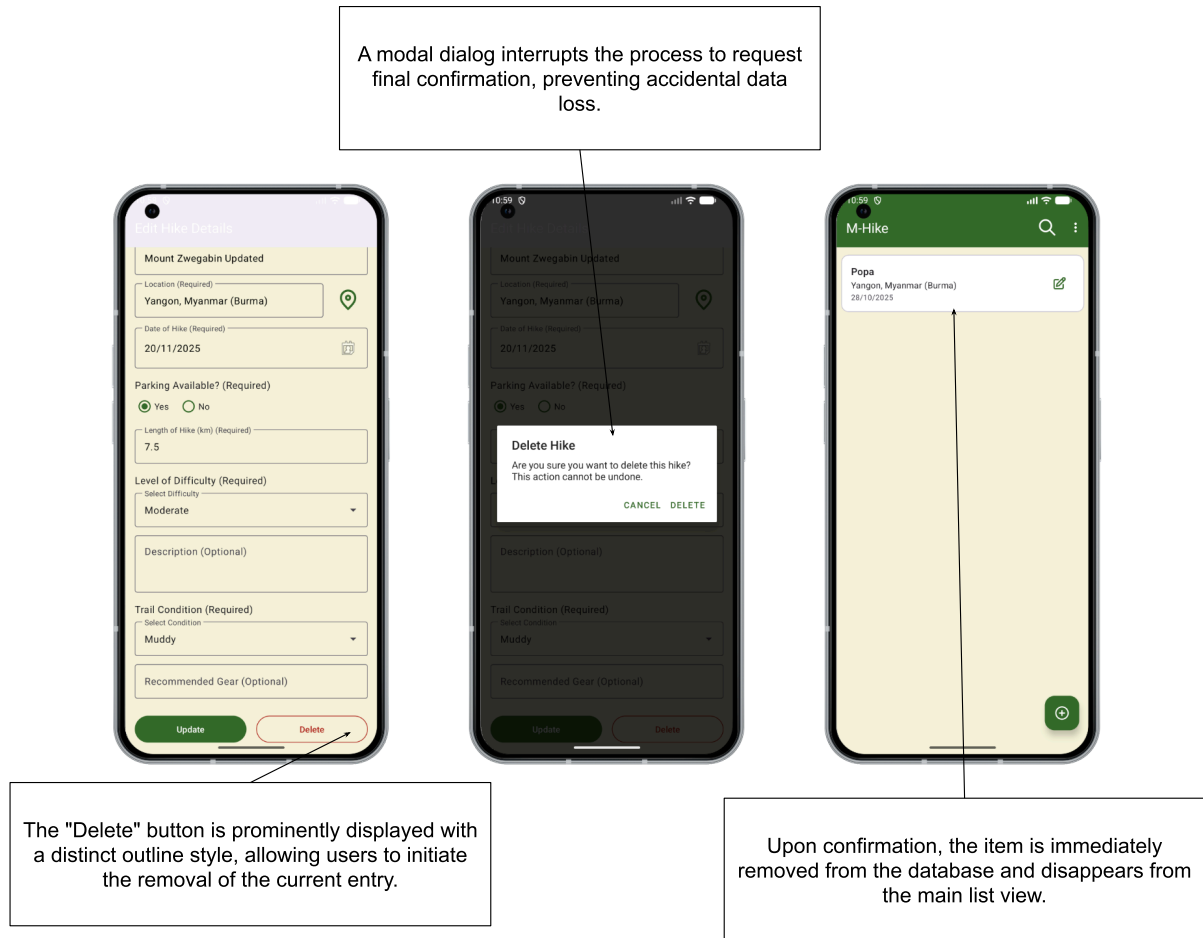
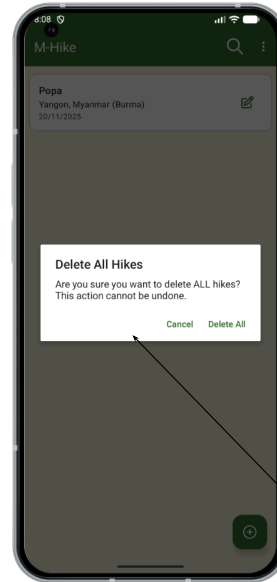
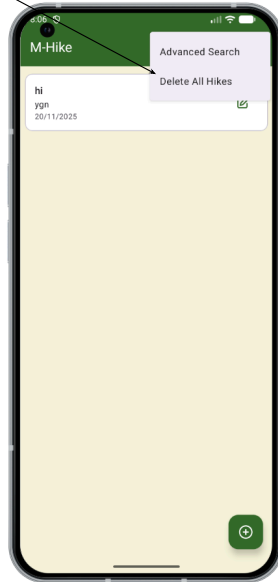


Figure 6: Edit & Delete Screen

The overflow menu in the App Bar provides access to global commands, allowing the user to initiate a complete database reset.

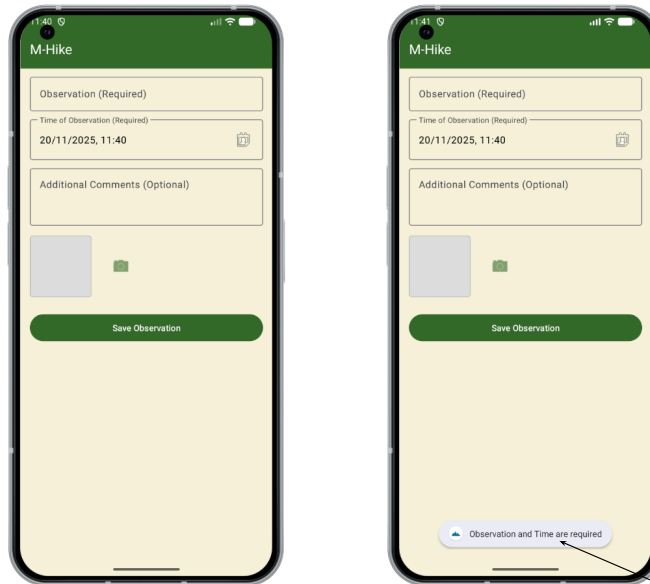


A modal alert interrupts the process to force a specific confirmation, preventing the accidental permanent loss of all data.

Figure 7: Delete All Hike Screen

Feature (C)

The interface combines standard text fields with a camera button, allowing users to seamlessly attach visual evidence to their notes.

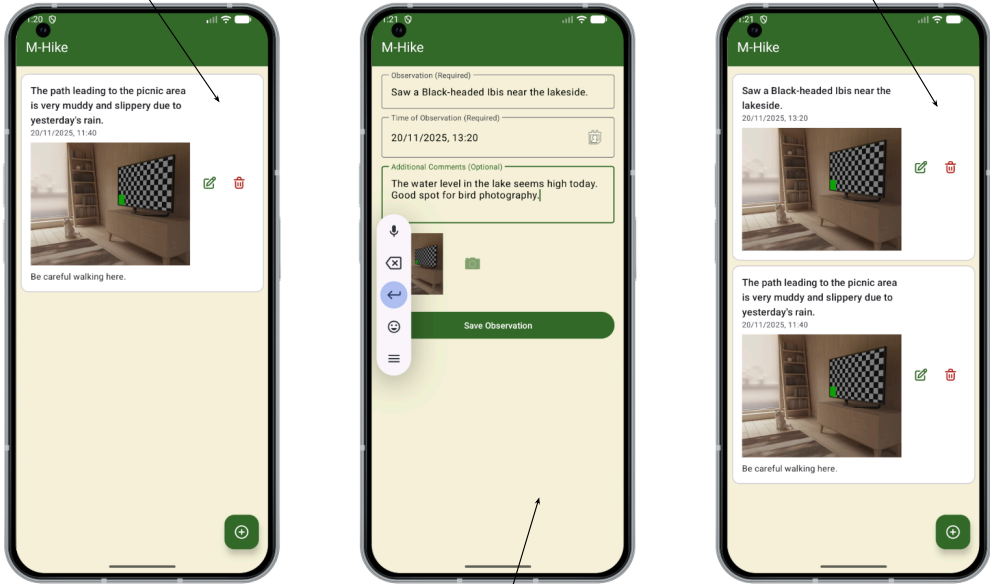


A toast message blocks the save action if mandatory fields are empty, strictly enforcing data integrity rules before submission.

Figure 8: Add Observation Screen

The list view presents previously recorded observations with their associated photos and timestamps, providing context before adding new information.

Upon saving, it is immediately added to the top of the list, visually confirming the successful update and maintaining a timeline of events.



The input screen allows the user to record specific details combined with photographic proof for a comprehensive field report.

Figure 9: Observation Screen

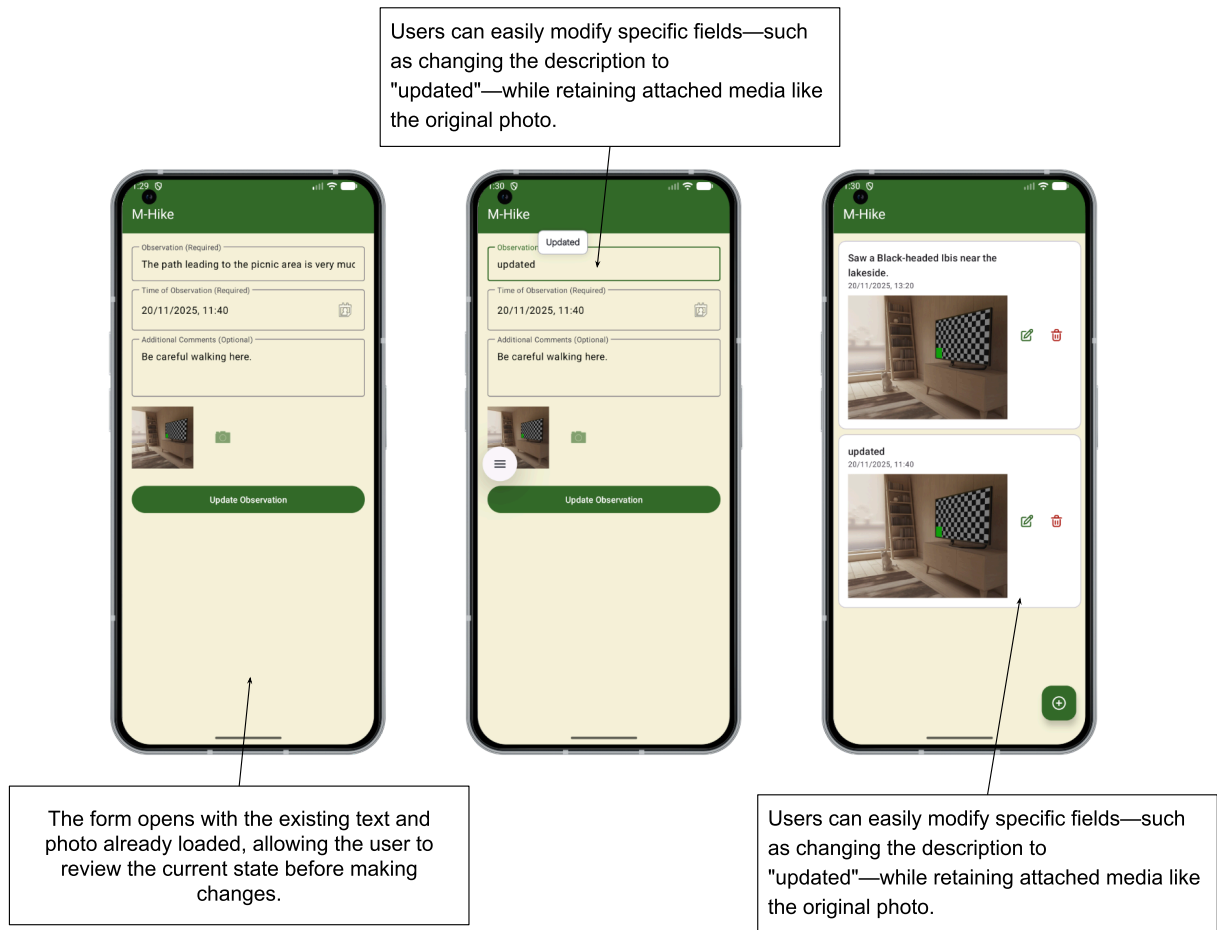
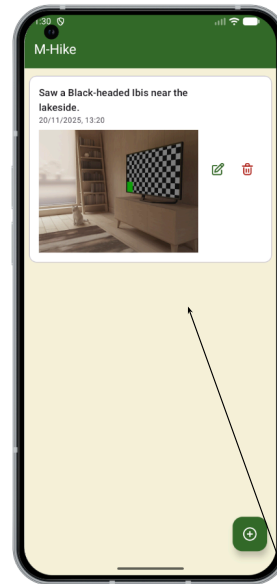
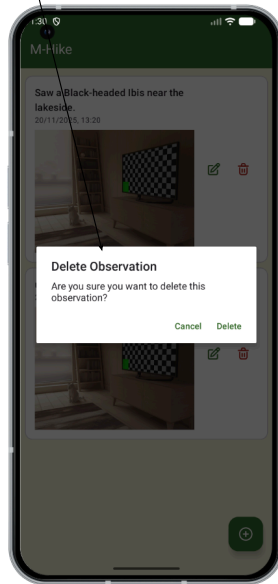


Figure 10: Edit Observation Screen

A modal dialog overlays the screen to request explicit confirmation, acting as a safeguard against the accidental deletion of a specific observation.



Once confirmed, the item is permanently removed from the database and instantly vanishes from the list view, reflecting the current data state.

Figure 11: Delete Observation Screen

Feature (D)

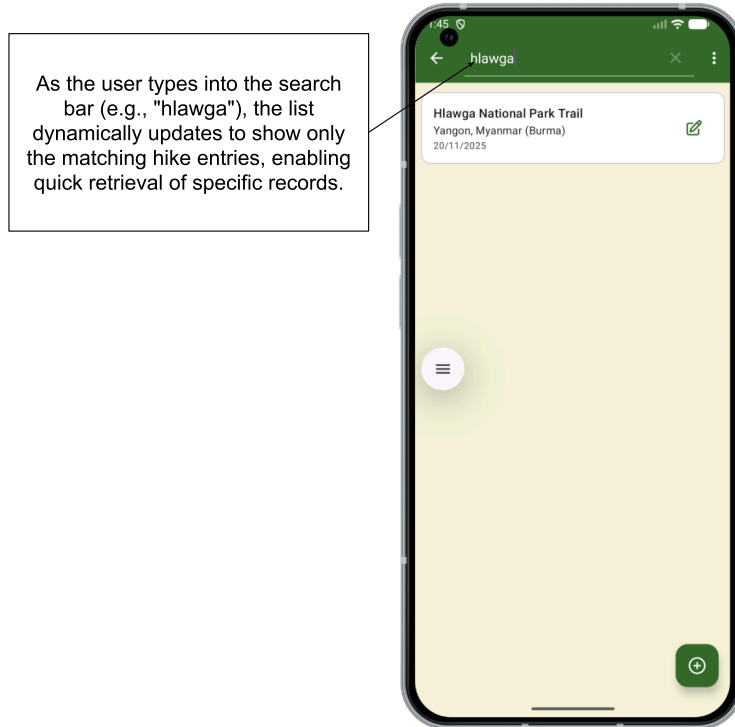


Figure 12: Search Screen

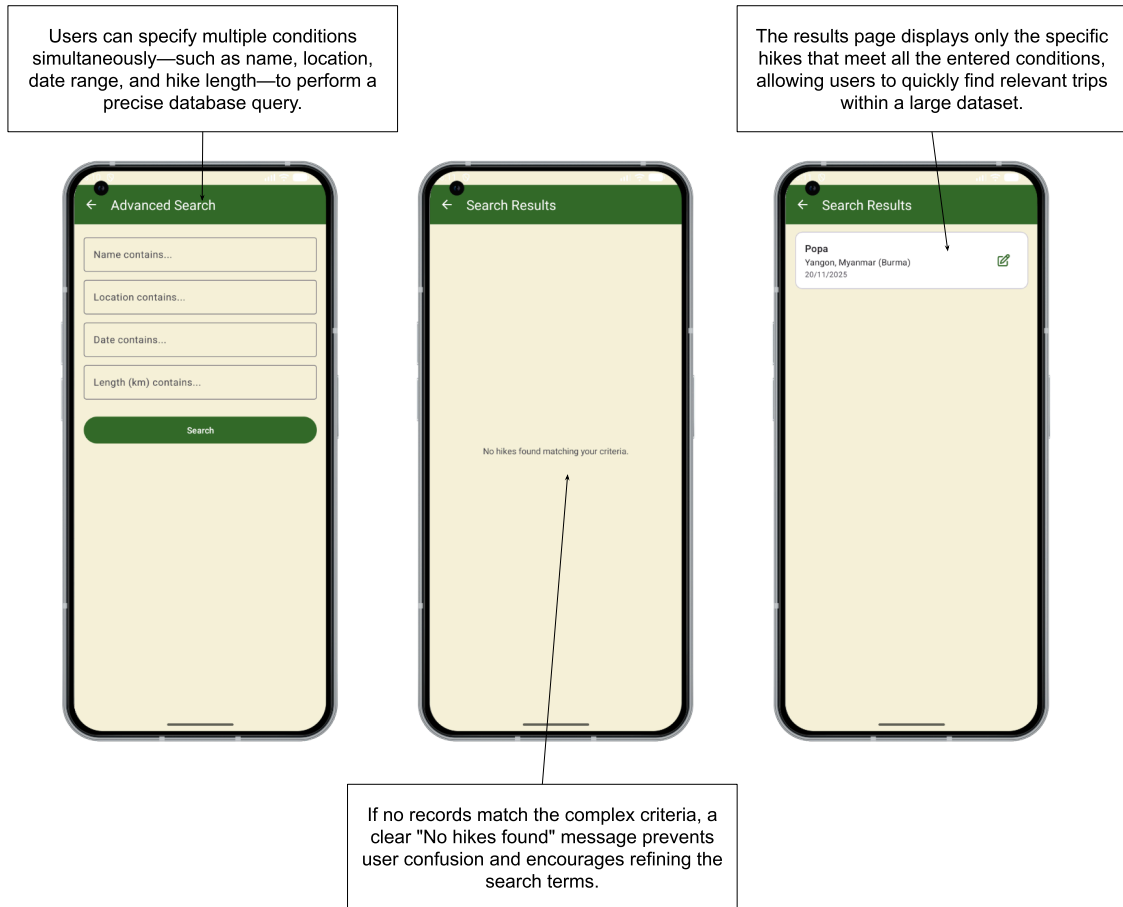


Figure 13: Advance Search Screen

Function (E)

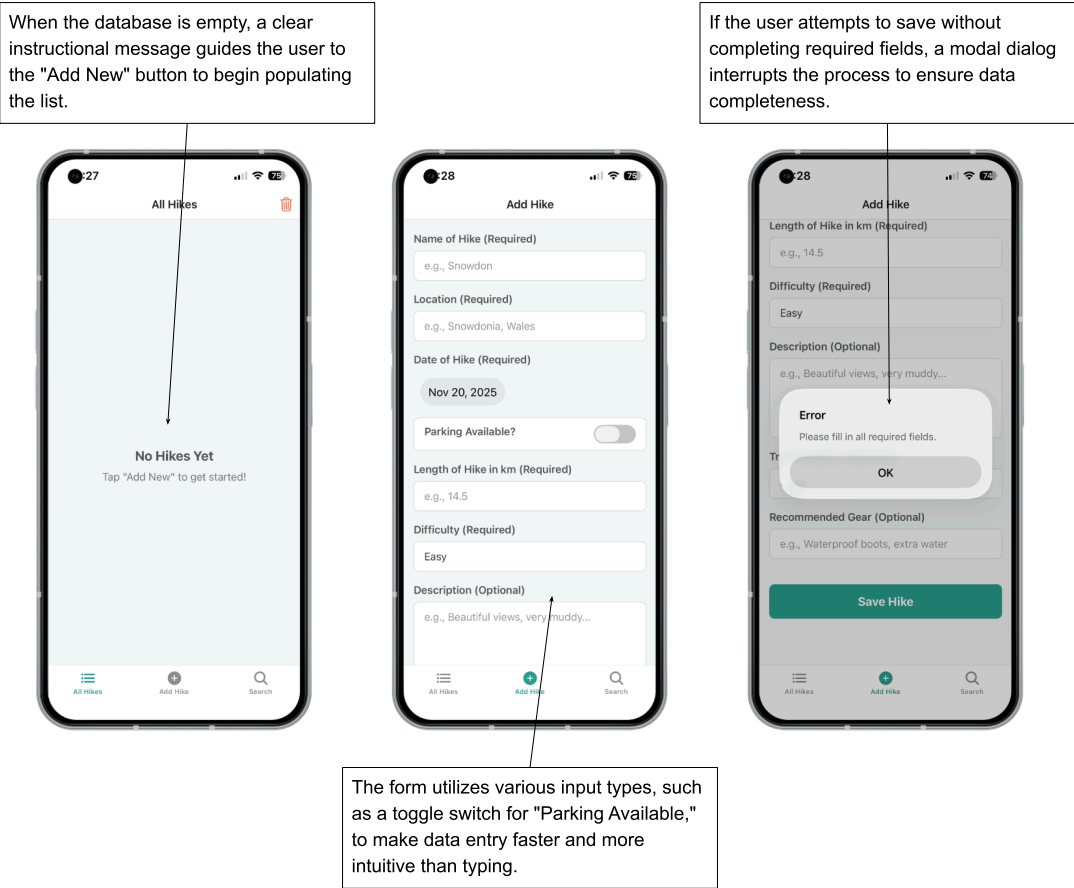
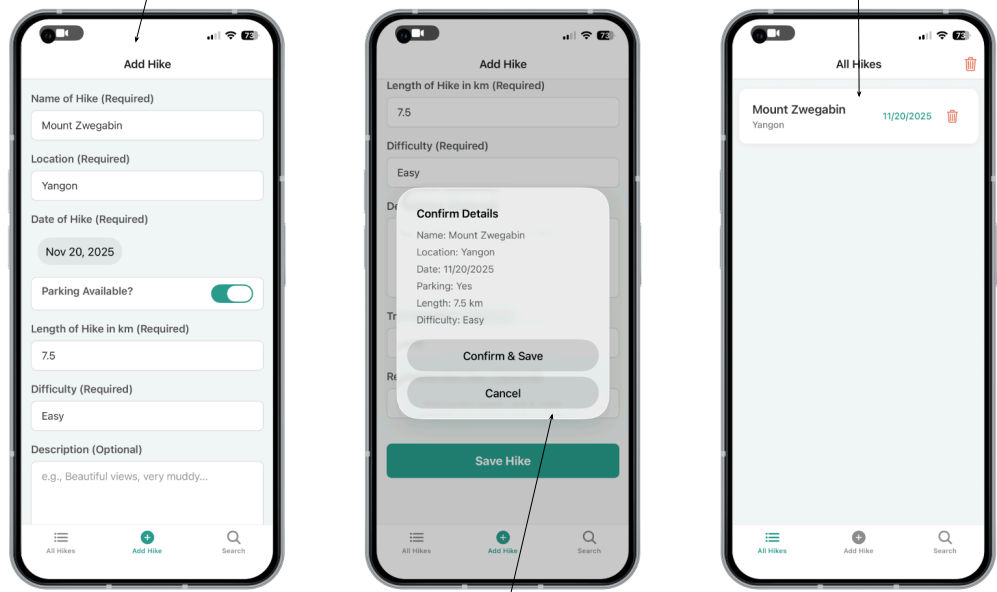


Figure 14: Home Screen

The user fills in specific details such as "Mount Zwegabin" and toggles settings like "Parking Available" to prepare the record for submission.

A summary modal interrupts the save process, forcing the user to review all entered data for accuracy before committing it to the database.



A summary modal interrupts the save process, forcing the user to review all entered data for accuracy before committing it to the database.

Figure 15: Add New Hike Screen

Function (F)

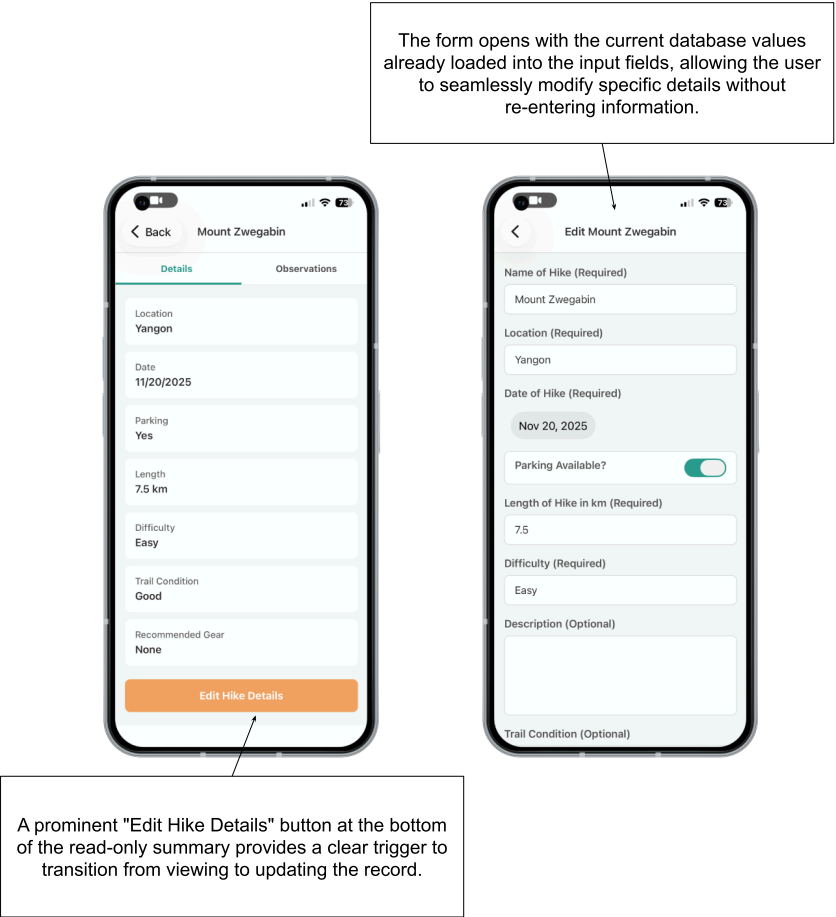


Figure 16: Detail Screen

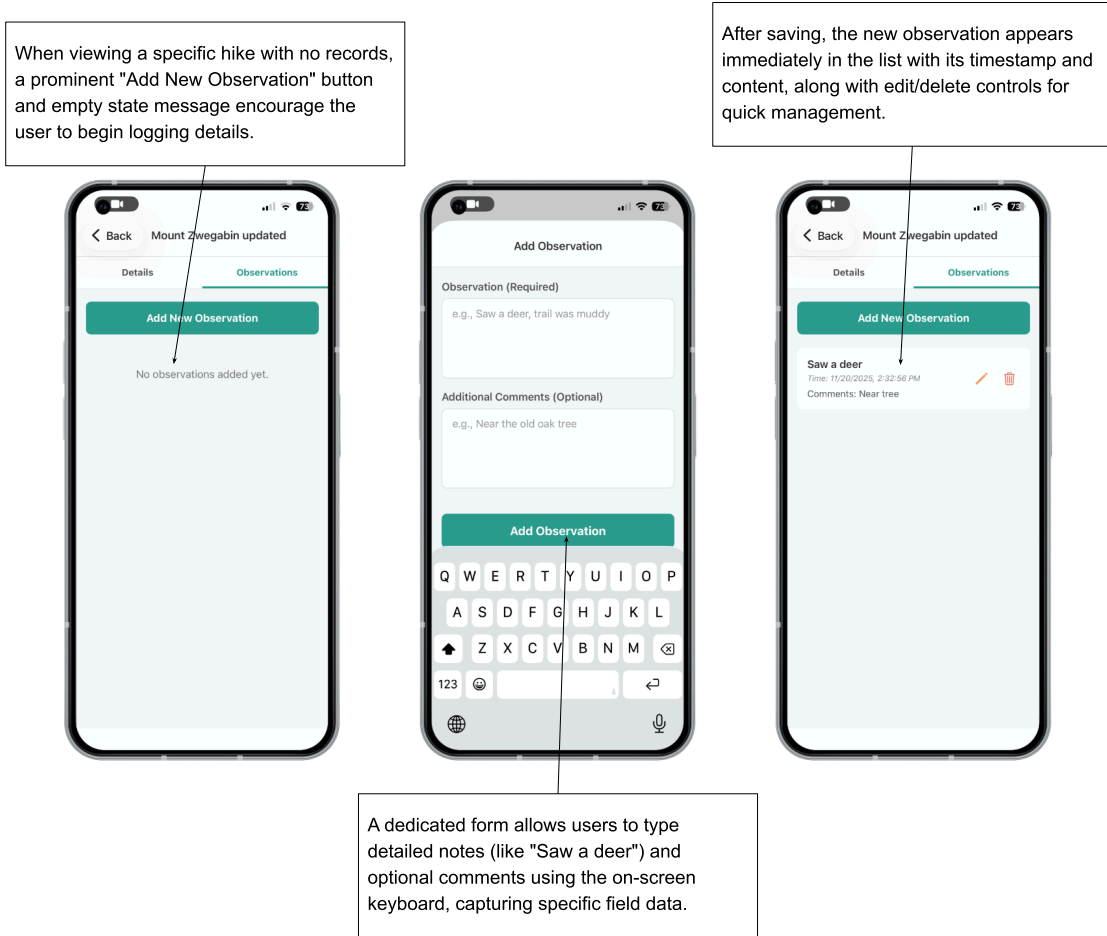
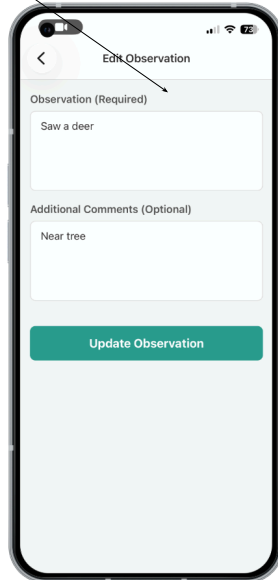
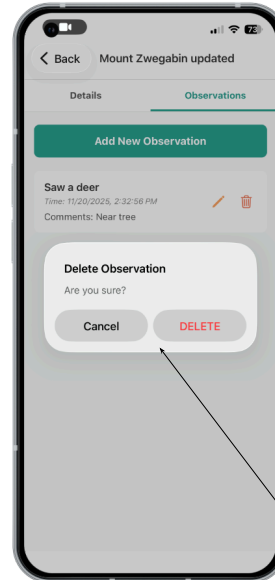


Figure 17: Add Observation Screen

Tapping the pencil icon opens this form pre-filled with existing data, allowing the user to revise specific details like the observation text.



The 'Edit Observation' screen features a back arrow at the top left. Below the title, there are two text input fields: 'Observation (Required)' with the placeholder text 'Saw a deer' and 'Additional Comments (Optional)' with the placeholder text 'Near tree'. A green 'Update Observation' button is positioned at the bottom of the form area.



The 'Delete Observation' modal dialog is displayed over a blurred background of the 'Observations' list. The dialog has a title 'Delete Observation' and the question 'Are you sure?'. It contains two buttons: a grey 'Cancel' button and a red 'DELETE' button.

Tapping the trash can icon triggers this modal dialog to verify the user's intent ("Are you sure?") before permanently removing the selected observation.

Figure 18: Edit & Delete Observation Screen

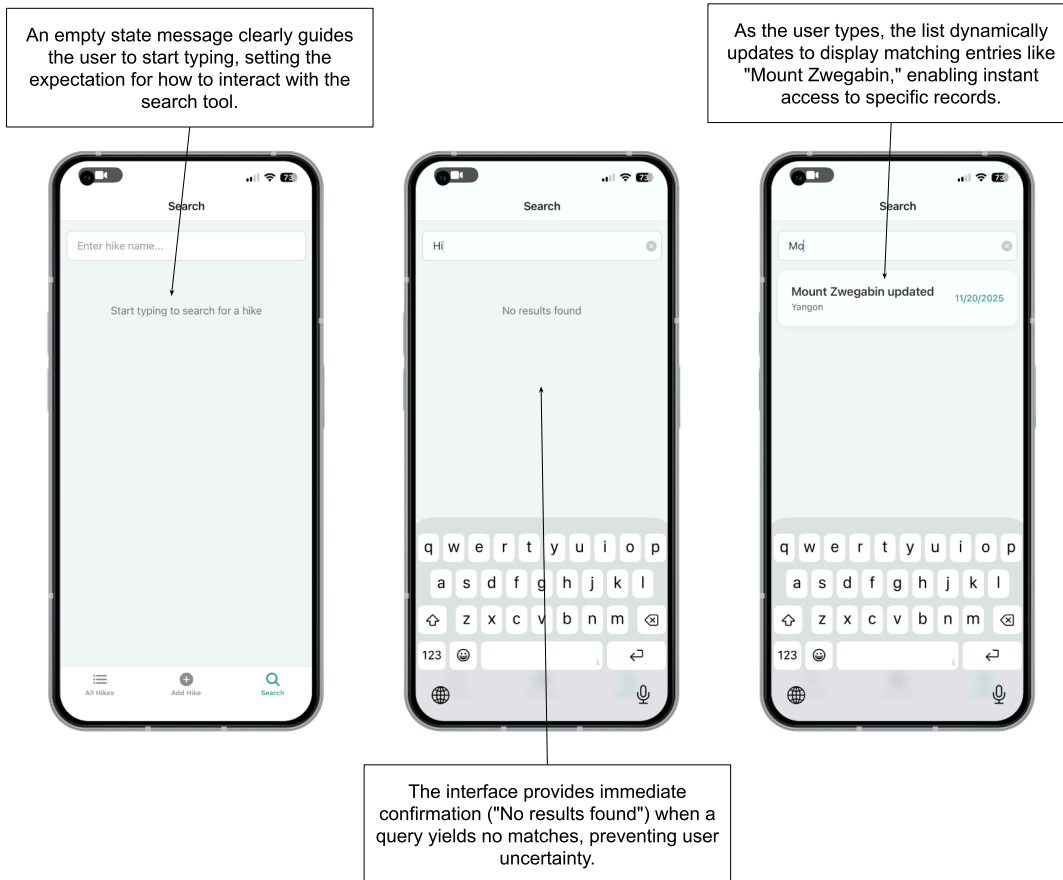
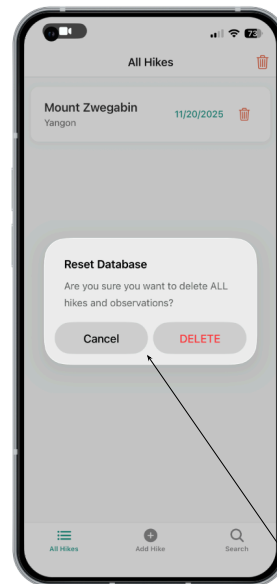
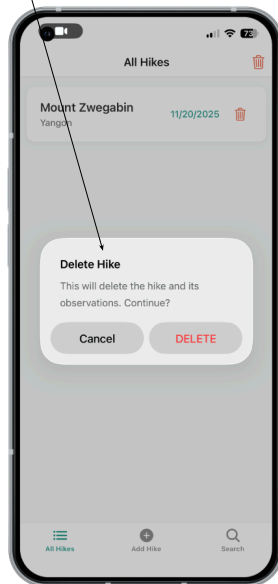


Figure 19: Search Screen

Triggered by clicking the trash icon on a **specific list card**, this dialog confirms the deletion of just **that one hike** and its notes.



Triggered by clicking the trash icon in the **top App Bar**, this dialog confirms a "Reset" action that permanently wipes **all** records from the application.

Figure 20: Delete Screen

Function (G)

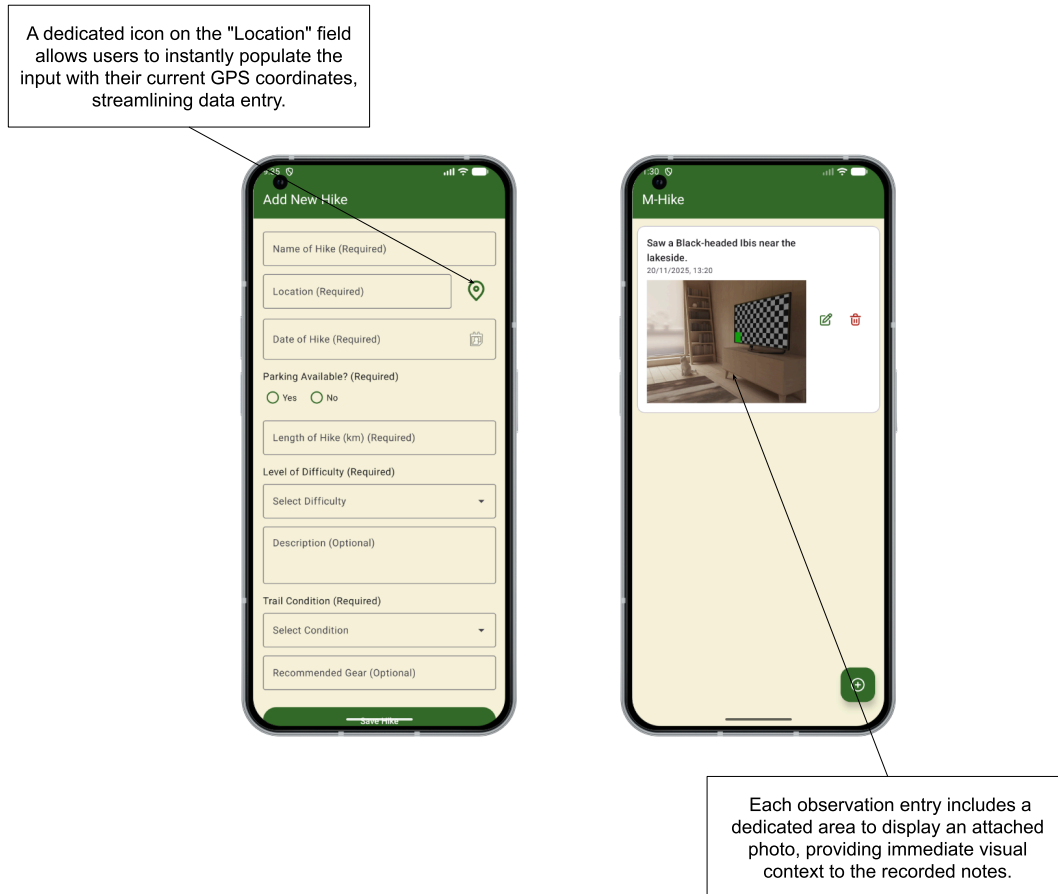


Figure 21: Advance Feature Screen

SECTION 3 - REFLECTION

The M-Hike application development was also a learning curve of transitioning to the modern Android development using Kotlin and Jetpack Architecture elements as opposed to basic Java principles. The main idea was to create a powerful data-logging application, and the choice to work with the Model-View-ViewModel (MVVM) framework and Room Database was critical.

The implementation of the database layer was one of the things that went well. Originally, I was thinking of using the older SQLiteOpenHelper, but the research on the industry standards led to Room. Room offered an abstraction layer that allowed defining Entities (tables) and DAOs (queries) far more intuitively and with fewer mistakes than directly writing SQL strings. In addition, the Kotlin Coroutines (suspend functions) were used, which facilitated the database operations to occur asynchronously off the primary thread and did not freeze the UI, which was an enormous improvement in comparison to the standard thread management.

Nevertheless, the process of development did not go without difficulties. A big challenge was that of applying the UI styling to the "Add Observation" screen. The XML appeared to be correct but the Material Design components were not rendering with the appropriate padding or outlines. By debugging I got to know the significance of themes.xml inheritance and interaction between a certain activity and the application theme. I also discovered that there is a possibility of the IDE cache creating so-called ghost bugs and that a Clean/Rebuild or an Invalidate Caches can be a required troubleshooting step in Android Studio.

The other area that was intricate was to deal with permissions to the advanced features. The introduction of the Camera and Location features needed to know the modern ActivityResultContracts and not the outdated startActivityForResult. Although the feature is currently functional, the treatment of the edge cases (e.g. in case a user denies permanently) may be better to offer a more positive user experience.

Reflectively, in case I were to re-initiate this project, I would have adopted a dependency injection framework such as Hilt at the onset of the project. Although the

manual creation of the Repository and ViewModel factories was viable in the case of an app of this scale, dependency injection would help in making the code cleaner and testable as the app grows. In general, the project is able to satisfy its requirements and the photo capabilities provide important add value to the user experience.

SECTION 4 - EVALUATION

Introduction

The M-Hike application was designed to serve as a digital logbook for hikers, allowing for the planning of trips and the recording of observations. The application meets all core functional requirements, including CRUD (Create, Read, Update, Delete) operations on hikes and observations, and includes advanced features such as geolocation and photo attachments. This evaluation critiques the application based on Human-Computer Interaction (HCI) principles, security, scalability, and deployment readiness.

Human-Computer Interaction (HCI)

The application design heavily relies on Google's Material Design 3 guidelines, ensuring a user interface that is familiar and intuitive to Android users.

- **Consistency and Standards:** The app utilizes standard navigation patterns. The use of a Floating Action Button (FAB) for the primary "Add" action in the top App Bar adheres to Jakob Nielsen's heuristic of consistency. Users do not need to wonder whether different words, situations, or actions mean the same thing.
- **Error Prevention and Recovery:** A key strength of the app is its proactive error prevention. The input forms utilize `TextInputLayout` components that provide visual cues (hints) about required fields. Furthermore, the "Delete" functionality is guarded by a confirmation dialog (an explicit "Safety Check"). This prevents the "slip" type of error where a user accidentally taps a destructive button.
- **Visibility of System Status:** When a hike is saved or updated, the app uses "Toast" messages to provide immediate feedback. Additionally, the use of `LiveData` ensures that the UI automatically refreshes when the underlying database changes, keeping the user's view consistent with the system state.
- **Improvement:** While the feedback is functional, it could be improved by using a `Snackbar` instead of a `Toast` for deletions. A `Snackbar` allows for an "Undo"

action, which is a superior HCI pattern to a simple confirmation dialog as it allows for smoother workflow while still offering a safety net.

Security

Security is a critical aspect of mobile applications, even for offline apps like M-Hike.

- **Data Storage:** Currently, the app uses a standard SQLite database via Room. On a non-rooted device, this data is sandboxed and accessible only by the M-Hike app. However, on a rooted device, or if the database file is exported, the data is in plain text.
- **Permissions:** The app follows the principle of "Least Privilege." It only requests Location and Camera permissions when the user explicitly interacts with those features (e.g., clicking the location button), rather than requesting them all at startup. This builds user trust.
- **Improvement:** To make the app more secure, the database should be encrypted. This could be achieved by using SQLCipher for Android, which wraps the SQLite database with 256-bit AES encryption. Additionally, ProGuard rules should be enabled to obfuscate the code, making it difficult for malicious actors to reverse-engineer the application logic or extract API keys if external services were used.

Screen Size Compatibility

The application was developed using ConstraintLayout and LinearLayout with weighted widths. This ensures that the layout is responsive; elements stretch to fill the available width regardless of the specific device density or resolution.

- **Current State:** On a standard phone (e.g., Pixel 6), the list view is a single column, and the detail view takes up the full screen. This works well for portrait orientation on phones.

- **Limitations:** On a tablet or a phone in landscape mode, the current design is inefficient. A long list of hikes stretched across a 10-inch screen leaves significant white space, and the user must navigate back and forth between the list and the details.
- **Improvement:** To improve this, the app should implement a Master-Detail flow using Fragments and a SlidingPaneLayout. On large screens (sw600dp), the app should display the list of hikes on the left (30% width) and the details of the selected hike on the right (70% width) simultaneously. This would significantly reduce the navigation effort (interaction cost) for tablet users.

Deployment Readiness

The current version of the app is a "Debug" build, intended for development. Several changes are required to prepare it for "Live" deployment on the Google Play Store:

- **Code Signing:** The app must be signed with a release Keystore. This cryptographic signature verifies the developer's identity and ensures that the APK has not been tampered with.
- **Build Configuration:** The build.gradle file needs to be updated to set minifyEnabled true and shrinkResources true. This activates R8 (the code shrinker), which removes unused code and resources, reducing the app's download size and obfuscating the code for security.
- **Logging:** Currently, the app may contain e.printStackTrace() or Toast messages used for debugging. These should be removed or wrapped in a conditional check (e.g., if (BuildConfig.DEBUG)). Professional logging tools like Firebase Crashlytics should be integrated to track crashes in the wild, as developers cannot see the logs of user devices.
- **Accessibility:** While the app uses standard controls, a full accessibility audit is needed. This involves ensuring all ImageButtons (like the edit/delete icons) have

meaningful contentDescription attributes for screen readers (TalkBack) and that the color contrast ratios meet WCAG 2.0 AA standards.

SECTION 5 - CODE

[Hike.kt](#)

```
@Parcelize
@Entity(tableName = "hikes")
data class Hike(
    @PrimaryKey(autoGenerate = true)
    val id: Long = 0,
    val name: String,
    val location: String,
    val date: String,
    val parking: String,
    val length: String,
    val difficulty: String,
    val description: String?,

    // My two custom fields
    val trailCondition: String,
    val recommendedGear: String?
) : Parcelable
```

[Observation.kt](#)

```
@Parcelize
@Entity(
    tableName = "observations",
    foreignKeys = [ForeignKey(
        entity = Hike::class,
        parentColumns = ["id"],
        childColumns = ["hikeId"],
        onDelete = ForeignKey.CASCADE
    )],

    indices = [Index(value = ["hikeId"])]
)
```

```

data class Observation(
    @PrimaryKey(autoGenerate = true)
    val id: Long = 0,
    val hikeId: Long,
    val observation: String,
    val time: String,
    val comments: String?,
    val photoUri: String?
) : Parcelable

```

[HikeDao.kt](#)

```

@Dao
interface HikeDao {

    @Insert
    suspend fun addHike(hike: Hike)

    @Update
    suspend fun updateHike(hike: Hike)

    @Delete
    suspend fun deleteHike(hike: Hike)

    @Query("DELETE FROM hikes")
    suspend fun deleteAllHikes()

    @Query("SELECT * FROM hikes ORDER BY id DESC")
    fun getAllHikes(): LiveData<List<Hike>>

    @Query("SELECT * FROM hikes WHERE name LIKE :query OR location LIKE :query")
    fun searchHikes(query: String): LiveData<List<Hike>>

    @Query("SELECT * FROM hikes WHERE id = :hikeId")
    fun getHike(hikeId: Long): LiveData<Hike>

```

```

    @Query("SELECT * FROM hikes WHERE name LIKE '%' || :name || '%'
AND location LIKE '%' || :location || '%' AND length LIKE '%' ||
:length || '%' AND date LIKE '%' || :date || '%' ORDER BY date DESC")
    fun advancedSearchHikes(name: String, location: String, length:
String, date: String): LiveData<List<Hike>>

    // --- Observation Actions ---

    @Insert
    suspend fun addObservation(observation: Observation)

    @Query("SELECT * FROM observations WHERE hikeId = :hikeId ORDER BY
time DESC")
    fun getObservationsForHike(hikeId: Long):
LiveData<List<Observation>>

    @Update
    suspend fun updateObservation(observation: Observation)

    @Delete
    suspend fun deleteObservation(observation: Observation)

}

```

[AppDatabase.kt](#)

```

@Database(entities = [Hike::class, Observation::class], version = 1,
exportSchema = false)
abstract class AppDatabase : RoomDatabase() {

    abstract fun hikeDao(): HikeDao

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        fun getDatabase(context: Context): AppDatabase {

```

```

        return INSTANCE ?: synchronized(this) {
            val instance = Room.databaseBuilder(
                context.applicationContext,
                AppDatabase::class.java,
                "m_hike_database"
            ).fallbackToDestructiveMigration().build()
            INSTANCE = instance
            instance
        }
    }
}

```

[HikeRepository.kt](#)

```

package com.example.mhike
import androidx.lifecycle.LiveData

class HikeRepository(private val hikeDao: HikeDao) {

    val allHikes: LiveData<List<Hike>> = hikeDao.getAllHikes()

    suspend fun addHike(hike: Hike) {
        hikeDao.addHike(hike)
    }

    suspend fun updateHike(hike: Hike) {
        hikeDao.updateHike(hike)
    }

    suspend fun deleteHike(hike: Hike) {
        hikeDao.deleteHike(hike)
    }

    suspend fun deleteAllHikes() {
        hikeDao.deleteAllHikes()
    }
}

```

```

    fun searchHikes(query: String): LiveData<List<Hike>> {
        return hikeDao.searchHikes(query)
    }

    fun getHike(id: Long): LiveData<Hike> {
        return hikeDao.getHike(id)
    }

    fun advancedSearchHikes(name: String, location: String, length:
String, date: String): LiveData<List<Hike>> {
        return hikeDao.advancedSearchHikes(name, location, length,
date)
    }

    // --- Observation Functions ---

    suspend fun addObservation(observation: Observation) {
        hikeDao.addObservation(observation)
    }

    fun getObservationsForHike(hikeId: Long):
LiveData<List<Observation>> {
        return hikeDao.getObservationsForHike(hikeId)
    }

    suspend fun updateObservation(observation: Observation) {
        hikeDao.updateObservation(observation)
    }

    suspend fun deleteObservation(observation: Observation) {
        hikeDao.deleteObservation(observation)
    }
}

```

[HikeViewModel.kt](#)

```

class HikeViewModel(application: Application) :
    AndroidViewModel(application) {

```

```
private val repository: HikeRepository
val allHikes: LiveData<List<Hike>>

init {
    val hikeDao = AppDatabase.getDatabase(application).hikeDao()
    repository = HikeRepository(hikeDao)
    allHikes = repository.allHikes
}

fun addHike(hike: Hike) = viewModelScope.launch(Dispatchers.IO) {
    repository.addHike(hike)
}

fun updateHike(hike: Hike) = viewModelScope.launch(Dispatchers.IO)
{
    repository.updateHike(hike)
}

fun deleteHike(hike: Hike) = viewModelScope.launch(Dispatchers.IO)
{
    repository.deleteHike(hike)
}

fun deleteAllHikes() = viewModelScope.launch(Dispatchers.IO) {
    repository.deleteAllHikes()
}

fun searchHikes(query: String): LiveData<List<Hike>> {
    return repository.searchHikes(query)
}

fun getHike(id: Long): LiveData<Hike> {
    return repository.getHike(id)
}

fun advancedSearchHikes(name: String, location: String, length:
String, date: String): LiveData<List<Hike>> {
    return repository.advancedSearchHikes(name, location, length,
```

```

date)
    }

    // --- Observation Functions ---

    fun addObservation(observation: Observation) =
        viewModelScope.launch(Dispatchers.IO) {
            repository.addObservation(observation)
        }

    fun getObservationsForHike(hikeId: Long):
        LiveData<List<Observation>> {
        return repository.getObservationsForHike(hikeId)
    }

    fun updateObservation(observation: Observation) =
        viewModelScope.launch(Dispatchers.IO) {
            repository.updateObservation(observation)
        }

    fun deleteObservation(observation: Observation) =
        viewModelScope.launch(Dispatchers.IO) {
            repository.deleteObservation(observation)
        }
}

```

[MainActivity.kt](#)

```

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding
    private lateinit var viewModel: HikeViewModel
    private lateinit var hikeAdapter: HikeAdapter

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
    }
}

```

```

        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        setSupportActionBar(binding.toolbar)

        viewModel =
ViewModelProvider(this).get(HikeViewModel::class.java)

        hikeAdapter = HikeAdapter(
            onCardClicked = { hike ->
                val intent = Intent(this,
HikeObservationActivity::class.java)
                intent.putExtra("HIKE_ID", hike.id)
                startActivity(intent)
            },
            onEditClicked = { hike ->
                val intent = Intent(this,
HikeDetailActivity::class.java)
                intent.putExtra("HIKE_ID", hike.id)
                startActivity(intent)
            }
        )

        binding.recyclerView.adapter = hikeAdapter

        viewModel.allHikes.observe(this, Observer { hikes ->
            hikes?.let {
                hikeAdapter.submitList(it)
            }
        })

        binding.fabAddHike.setOnClickListener {
            val intent = Intent(this, AddHikeActivity::class.java)
            startActivity(intent)
        }
    }

    override fun onCreateOptionsMenu(menu: Menu): Boolean {
        menuInflater.inflate(R.menu.main_menu, menu)
    }

```

```

        val searchItem = menu.findItem(R.id.action_search)
        val searchView = searchItem?.actionView as? SearchView

        if (searchView != null) {
            searchView.setOnQueryTextListener(object :
                SearchView.OnQueryTextListener {
                    override fun onQueryTextSubmit(query: String?):
                        Boolean {
                            query?.let { performSearch(it) }
                            return true
                        }

                    override fun onQueryTextChange(newText: String?):
                        Boolean {
                            newText?.let { performSearch(it) }
                            return true
                        }
                })
        }

        return true
    }

    override fun onOptionsItemSelected(item: MenuItem): Boolean {
        return when (item.itemId) {
            R.id.action_delete_all -> {
                showDeleteAllConfirmationDialog()
                true
            }
            R.id.action_advanced_search -> {
                val intent = Intent(this,
                    AdvancedSearchActivity::class.java)
                startActivity(intent)
                true
            }
            else -> super.onOptionsItemSelected(item)
        }
    }
}

```

```

private fun performSearch(query: String) {
    val searchQuery = "%$query%"

    viewModel.searchHikes(searchQuery).observe(this, Observer {
hikes ->
        hikes?.let {
            hikeAdapter.submitList(it)
        }
    })
}

private fun showDeleteAllConfirmationDialog() {
    AlertDialog.Builder(this)
        .setTitle("Delete All Hikes")
        .setMessage("Are you sure you want to delete ALL hikes?
This action cannot be undone.")
        .setPositiveButton("Delete All") { _, _ ->
            viewModel.deleteAllHikes()
            Toast.makeText(this, "All hikes deleted",
Toast.LENGTH_SHORT).show()
        }
        .setNegativeButton("Cancel", null)
        .show()
}
}

```

[AddHikeActivity.kt](#)

```

class AddHikeActivity : AppCompatActivity() {

    private lateinit var binding: ActivityAddHikeBinding
    private lateinit var viewModel: HikeViewModel
    private lateinit var fusedLocationClient:
FusedLocationProviderClient

    private val requestPermissionLauncher =

```

```

registerForActivityResult(ActivityResultContracts.RequestPermission()) { isGranted: Boolean ->
    if (isGranted) {
        getCurrentLocation()
    } else {
        Toast.makeText(this, "Location permission denied",
            Toast.LENGTH_SHORT).show()
    }
}

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = ActivityAddHikeBinding.inflate(layoutInflater)
    setContentView(binding.root)

    setSupportActionBar(binding.toolbar)
    supportActionBar?.setDisplayHomeAsUpEnabled(true)

    viewModel =
        ViewModelProvider(this).get(HikeViewModel::class.java)
    fusedLocationClient =
        LocationServices.getFusedLocationProviderClient(this)

    setupDatePicker()
    setupSpinners()

    binding.buttonGetLocation.setOnClickListener {
        checkLocationPermission()
    }

    binding.buttonSave.setOnClickListener {
        if (validateInput()) {
            showConfirmationDialog()
        }
    }
}

override fun onSupportNavigateUp(): Boolean {
    finish()
}

```

```

        return true
    }

    private fun setupDatePicker() {
        binding.editTextDate.setOnClickListener {
            val calendar = Calendar.getInstance()
            val year = calendar.get(Calendar.YEAR)
            val month = calendar.get(Calendar.MONTH)
            val day = calendar.get(Calendar.DAY_OF_MONTH)

            DatePickerDialog(this, { _, selectedYear, selectedMonth,
selectedDay ->
                val date = "$selectedDay/${selectedMonth +
1}/${selectedYear}"
                binding.editTextDate.setText(date)
            }, year, month, day).show()
        }
    }

    private fun setupSpinners() {
        val difficultyLevels =
resources.getStringArray(R.array.difficulty_levels)
        val difficultyAdapter = ArrayAdapter(this,
android.R.layout.simple_dropdown_item_1line, difficultyLevels)

        binding.spinnerDifficultyAutocomplete.setAdapter(difficultyAdapter)

        val trailConditions =
resources.getStringArray(R.array.trail_conditions)
        val conditionAdapter = ArrayAdapter(this,
android.R.layout.simple_dropdown_item_1line, trailConditions)

        binding.spinnerTrailConditionAutocomplete.setAdapter(conditionAdapter
        )
    }

    private fun validateInput(): Boolean {
        if (binding.editTextName.text.isNullOrEmpty()) {
            binding.editTextName.error = "Name is required"
        }
    }

```

```

        return false
    }
    if (binding.editTextLocation.text.isNullOrEmpty()) {
        binding.editTextLocation.error = "Location is required"
        return false
    }
    if (binding.editTextDate.text.isNullOrEmpty()) {
        binding.editTextDate.error = "Date is required"
        return false
    }
    if (binding.radioGroupParking.checkedRadioButtonId == -1) {
        Toast.makeText(this, "Please select parking availability",
Toast.LENGTH_SHORT).show()
        return false
    }
    if (binding.editTextLength.text.isNullOrEmpty()) {
        binding.editTextLength.error = "Length is required"
        return false
    }
    return true
}

private fun showConfirmationDialog() {
    val name = binding.editTextName.text.toString()
    val location = binding.editTextLocation.text.toString()
    val date = binding.editTextDate.text.toString()
    val parking = if (binding.radioParkingYes.isChecked) "Yes"
else "No"
    val length = binding.editTextLength.text.toString()
    val difficulty =
binding.spinnerDifficultyAutocomplete.text.toString()
    val trailCondition =
binding.spinnerTrailConditionAutocomplete.text.toString()

    val message = ""
        Name: $name
        Location: $location
        Date: $date
        Parking: $parking

```

```

        Length: $length km
        Difficulty: $difficulty
        Trail Condition: $trailCondition
    """.trimIndent()

    AlertDialog.Builder(this)
        .setTitle("Confirm Hike Details")
        .setMessage(message)
        .setPositiveButton("Confirm") { _, _ ->
            saveHike()
        }
        .setNegativeButton("Edit") { dialog, _ ->
            dialog.dismiss()
        }
        .show()
}

private fun saveHike() {
    val hike = Hike(
        name = binding.editTextName.text.toString(),
        location = binding.editTextLocation.text.toString(),
        date = binding.editTextDate.text.toString(),
        parking = if (binding.radioParkingYes.isChecked) "Yes"
else "No",
        length = binding.editTextLength.text.toString(),
        difficulty =
binding.spinnerDifficultyAutocomplete.text.toString(),
        description = binding.editTextDescription.text.toString(),
        trailCondition =
binding.spinnerTrailConditionAutocomplete.text.toString(),
        recommendedGear = binding.editTextGear.text.toString()
    )

    viewModel.addHike(hike)
    Toast.makeText(this, "Hike saved successfully!",
Toast.LENGTH_SHORT).show()
    finish()
}

```

```

private fun checkLocationPermission() {
    when {
        ContextCompat.checkSelfPermission(
            this,
            Manifest.permission.ACCESS_FINE_LOCATION
        ) == PackageManager.PERMISSION_GRANTED -> {
            getCurrentLocation()
        }
    }

    shouldShowRequestPermissionRationale(Manifest.permission.ACCESS_FINE_LOCATION) -> {
        AlertDialog.Builder(this)
            .setTitle("Location Permission Needed")
            .setMessage("This app needs the location permission to automatically fill the location field.")
            .setPositiveButton("OK") { _, _ ->

requestPermissionLauncher.launch(Manifest.permission.ACCESS_FINE_LOCATION)

            }
            .setNegativeButton("Cancel", null)
            .show()
        }
    }
    else -> {

requestPermissionLauncher.launch(Manifest.permission.ACCESS_FINE_LOCATION)

        }
    }
}

@SuppressLint("MissingPermission")
private fun getCurrentLocation() {
    if (ContextCompat.checkSelfPermission(this,
Manifest.permission.ACCESS_FINE_LOCATION) !=
PackageManager.PERMISSION_GRANTED &&
        ContextCompat.checkSelfPermission(this,
Manifest.permission.ACCESS_COARSE_LOCATION) !=
PackageManager.PERMISSION_GRANTED) {

```

```

        return
    }

fusedLocationClient.getCurrentLocation(Priority.PRIORITY_HIGH_ACCURACY, null)
    .addOnSuccessListener { location ->
        if (location != null) {
            val geocoder = Geocoder(this, Locale.getDefault())
            try {
                val addresses =
geocoder.getFromLocation(location.latitude, location.longitude, 1)
                if (addresses != null &&
addresses.isNotEmpty()) {
                    val address = addresses[0]
                    val locationText = "${address.locality ?:
"", ${address.countryName ?: ""}"

binding.editTextLocation.setText(locationText)
                } else {
                    Toast.makeText(this, "Could not find
address", Toast.LENGTH_SHORT).show()
                }
            } catch (e: Exception) {
                Toast.makeText(this, "Error getting address:
${e.message}", Toast.LENGTH_SHORT).show()
            }
        } else {
            Toast.makeText(this, "Could not get current
location. Is GPS on?", Toast.LENGTH_SHORT).show()
        }
    }
    .addOnFailureListener {
        Toast.makeText(this, "Failed to get location:
${it.message}", Toast.LENGTH_SHORT).show()
    }
}
}

```

[HikeDetailActivity.kt](#)

```
class HikeDetailActivity : AppCompatActivity() {

    private lateinit var binding: ActivityHikeDetailBinding
    private lateinit var viewModel: HikeViewModel
    private var hikeId: Long = -1
    private var currentHike: Hike? = null

    private lateinit var fusedLocationClient:
FusedLocationProviderClient
    private val requestPermissionLauncher =

registerForActivityResult(ActivityResultContracts.RequestPermission()
) { isGranted: Boolean ->
    if (isGranted) {
        getCurrentLocation()
    } else {
        Toast.makeText(this, "Location permission denied",
Toast.LENGTH_SHORT).show()
    }
}

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityHikeDetailBinding.inflate(layoutInflater)
        setContentView(binding.root)

        setSupportActionBar(binding.toolbar)
        supportActionBar?.setDisplayHomeAsUpEnabled(true)

        viewModel =
ViewModelProvider(this).get(HikeViewModel::class.java)

        hikeId = intent.getLongExtra("HIKE_ID", -1)

        if (hikeId == -1L) {
            Toast.makeText(this, "Error: Hike ID not found",
Toast.LENGTH_SHORT).show()
        }
    }
}
```

```

        finish()
        return
    }

    fusedLocationClient =
LocationServices.getFusedLocationProviderClient(this)

    setupSpinners()
    setupDatePicker()

    viewModel.getHike(hikeId).observe(this, Observer { hike ->
        hike?.let {
            currentHike = it
            populateForm(it)
        }
    })

    setupButtons()
}

override fun onSupportNavigateUp(): Boolean {
    finish()
    return true
}

private fun populateForm(hike: Hike) {
    binding.editTextName.setText(hike.name)
    binding.editTextLocation.setText(hike.location)
    binding.editTextDate.setText(hike.date)
    binding.editTextLength.setText(hike.length)
    binding.editTextDescription.setText(hike.description)
    binding.editTextGear.setText(hike.recommendedGear)

    if (hike.parking == "Yes") {
        binding.radioParkingYes.isChecked = true
    } else {
        binding.radioParkingNo.isChecked = true
    }
}

```

```

binding.spinnerDifficultyAutocomplete.setText(hike.difficulty, false)

binding.spinnerTrailConditionAutocomplete.setText(hike.trailCondition
, false)
    }

    private fun setupDatePicker() {
        binding.editTextDate.setOnClickListener {
            val calendar = Calendar.getInstance()
            val year = calendar.get(Calendar.YEAR)
            val month = calendar.get(Calendar.MONTH)
            val day = calendar.get(Calendar.DAY_OF_MONTH)

            DatePickerDialog(this, { _, selectedYear, selectedMonth,
selectedDay ->
                val date = "$selectedDay/${selectedMonth +
1}/${selectedYear}"
                binding.editTextDate.setText(date)
            }, year, month, day).show()
        }
    }

    private fun setupSpinners() {
        val difficultyLevels =
resources.getStringArray(R.array.difficulty_levels)
        val difficultyAdapter = ArrayAdapter(this,
android.R.layout.simple_dropdown_item_1line, difficultyLevels)

binding.spinnerDifficultyAutocomplete.setAdapter(difficultyAdapter)

        val trailConditions =
resources.getStringArray(R.array.trail_conditions)
        val conditionAdapter = ArrayAdapter(this,
android.R.layout.simple_dropdown_item_1line, trailConditions)

binding.spinnerTrailConditionAutocomplete.setAdapter(conditionAdapter
)
    }

```

```

private fun setupButtons() {
    binding.buttonUpdate.setOnClickListener {
        if (validateInput()) {
            updateHike()
        }
    }

    binding.buttonDelete.setOnClickListener {
        showDeleteConfirmationDialog()
    }

    binding.buttonGetLocation.setOnClickListener {
        checkLocationPermission()
    }
}

private fun validateInput(): Boolean {
    if (binding.editTextName.text.isNullOrEmpty()) {
        binding.editTextName.error = "Name is required"
        return false
    }
    if (binding.editTextLocation.text.isNullOrEmpty()) {
        binding.editTextLocation.error = "Location is required"
        return false
    }
    if (binding.editTextDate.text.isNullOrEmpty()) {
        binding.editTextDate.error = "Date is required"
        return false
    }
    if (binding.radioGroupParking.checkedRadioButtonId == -1) {
        Toast.makeText(this, "Please select parking
availability", Toast.LENGTH_SHORT).show()
        return false
    }
    if (binding.editTextLength.text.isNullOrEmpty()) {
        binding.editTextLength.error = "Length is required"
        return false
    }
}

```

```

        return true
    }

    private fun updateHike() {
        val updatedHike = Hike(
            id = hikeId,
            name = binding.editTextName.text.toString(),
            location = binding.editTextLocation.text.toString(),
            date = binding.editTextDate.text.toString(),
            parking = if (binding.radioParkingYes.isChecked) "Yes"
else "No",
            length = binding.editTextLength.text.toString(),
            description =
binding.editTextDescription.text.toString(),
            recommendedGear = binding.editTextGear.text.toString(),
            difficulty =
binding.spinnerDifficultyAutocomplete.text.toString(),
            trailCondition =
binding.spinnerTrailConditionAutocomplete.text.toString(),
        )

        viewModel.updateHike(updatedHike)
        Toast.makeText(this, "Hike updated!",
Toast.LENGTH_SHORT).show()
        finish()
    }

    private fun showDeleteConfirmationDialog() {
        AlertDialog.Builder(this)
            .setTitle("Delete Hike")
            .setMessage("Are you sure you want to delete this hike?
This action cannot be undone.")
            .setPositiveButton("Delete") { _, _ ->
                currentHike?.let {
                    viewModel.deleteHike(it)
                    Toast.makeText(this, "Hike deleted",
Toast.LENGTH_SHORT).show()
                    finish()
                }
            }
    }

```

```

        }
        .setNegativeButton("Cancel", null)
        .show()
    }

    private fun checkLocationPermission() {
        when {
            ContextCompat.checkSelfPermission(
                this, Manifest.permission.ACCESS_FINE_LOCATION
            ) == PackageManager.PERMISSION_GRANTED -> {
                getLocation()
            }
        }

        shouldShowRequestPermissionRationale(Manifest.permission.ACCESS_FINE_LOCATION) -> {
            AlertDialog.Builder(this)
                .setTitle("Location Permission Needed")
                .setMessage("This app needs the location permission to automatically fill the location field.")
                .setPositiveButton("OK") { _, _ ->

                    requestPermissionLauncher.launch(Manifest.permission.ACCESS_FINE_LOCATION)

                }
                .setNegativeButton("Cancel", null)
                .show()
        }
        else -> {

            requestPermissionLauncher.launch(Manifest.permission.ACCESS_FINE_LOCATION)

        }
    }

    @SuppressWarnings("MissingPermission")
    private fun getLocation() {
        if (ContextCompat.checkSelfPermission(this, Manifest.permission.ACCESS_FINE_LOCATION) !=

```

```

PackageManager.PERMISSION_GRANTED &&
        ContextCompat.checkSelfPermission(this,
Manifest.permission.ACCESS_COARSE_LOCATION) !=
PackageManager.PERMISSION_GRANTED) {
            return
        }

fusedLocationClient.getCurrentLocation(Priority.PRIORITY_HIGH_ACCURAC
Y, null)
        .addOnSuccessListener { location ->
            if (location != null) {
                val geocoder = Geocoder(this,
Locale.getDefault())
                try {
                    val addresses =
geocoder.getFromLocation(location.latitude, location.longitude, 1)
                    if (addresses != null &&
addresses.isNotEmpty()) {
                        val address = addresses[0]
                        val locationText = "${address.locality ?
: ""}, ${address.countryName ?: ""}"

binding.editTextLocation.setText(locationText)
                    } else {
                        Toast.makeText(this, "Could not find
address", Toast.LENGTH_SHORT).show()
                    }
                } catch (e: Exception) {
                    Toast.makeText(this, "Error getting address:
${e.message}", Toast.LENGTH_SHORT).show()
                }
            } else {
                Toast.makeText(this, "Could not get current
location. Is GPS on?", Toast.LENGTH_SHORT).show()
            }
        }
        .addOnFailureListener {

```

```

        Toast.makeText(this, "Failed to get location:
${it.message}", Toast.LENGTH_SHORT).show()
    }
}
}

```

[HikeObservationActivity.kt](#)

```

class HikeObservationActivity : AppCompatActivity() {

    private lateinit var binding: ActivityHikeObservationBinding
    private lateinit var viewModel: HikeViewModel
    private lateinit var observationAdapter: ObservationAdapter
    private var hikeId: Long = -1

    private fun showDeleteObservationDialog(observation: Observation)
    {
        AlertDialog.Builder(this)
            .setTitle("Delete Observation")
            .setMessage("Are you sure you want to delete this
observation?")
            .setPositiveButton("Delete") { _, _ ->
                viewModel.deleteObservation(observation)
                Toast.makeText(this, "Observation deleted",
Toast.LENGTH_SHORT).show()
            }
            .setNegativeButton("Cancel", null)
            .show()
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding =
ActivityHikeObservationBinding.inflate(layoutInflater)
        setContentView(binding.root)

        // Get the hike ID from the intent
        hikeId = intent.getLongExtra("HIKE_ID", -1)
    }
}

```

```

        if (hikeId == -1L) {
            finish()
            return
        }

        viewModel =
ViewModelProvider(this).get(HikeViewModel::class.java)

// Setup the adapter and RecyclerView
        observationAdapter = ObservationAdapter(

            onCardClicked = {

            },

            onEditClicked = { observation ->
                val intent = Intent(this,
AddEditObservationActivity::class.java)
                intent.putExtra("OBSERVATION_OBJECT", observation) //
Pass the whole object
                intent.putExtra("OBSERVATION_ID", observation.id) //
Pass the ID
                startActivity(intent)
            },

            onDeleteClicked = { observation ->
                showDeleteObservationDialog(observation)
            }
        )
        binding.recyclerViewObservations.adapter = observationAdapter
        binding.recyclerViewObservations.layoutManager =
LinearLayoutManager(this)

// Observe the observations
        viewModel.getObservationsForHike(hikeId).observe(this,
Observer { observations ->
            observations?.let {
                observationAdapter.submitList(it)
            }
        })

```

```

    })

    // Set title (optional, but nice)
    viewModel.getHike(hikeId).observe(this, Observer { hike ->
        hike?.let {
            supportActionBar?.title = "Observations for
${it.name}"
        }
    })

    // FAB click listener
    binding.fabAddObservation.setOnClickListener {
        val intent = Intent(this,
AddEditObservationActivity::class.java)
        intent.putExtra("HIKE_ID", hikeId) // "Add Mode"
        startActivity(intent)
    }
}
}

```

[AddEditObservationActivity.kt](#)

```

class AddEditObservationActivity : AppCompatActivity() {

    private lateinit var binding: ActivityAddEditObservationBinding
    private lateinit var viewModel: HikeViewModel

    private var hikeId: Long = -1
    private var observationId: Long = -1
    private var currentObservation: Observation? = null
    private var isEditMode = false
    private val calendar: Calendar = Calendar.getInstance()

    private var photoUri: Uri? = null
    private var currentPhotoUriString: String? = null

    private val takePictureLauncher =
registerForActivityResult(ActivityResultContracts.TakePicture()) {

```

```

isSuccess ->
    if (isSuccess) {
        currentPhotoUriString = photoUri.toString()
        binding.formContent.imageViewObservation.load(photoUri)
    }
}

private val cameraPermissionLauncher =
registerForActivityResult(ActivityResultContracts.RequestPermission()
) { isGranted ->
    if (isGranted) {
        launchCamera()
    } else {
        Toast.makeText(this, "Camera permission denied",
Toast.LENGTH_SHORT).show()
    }
}

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding =
ActivityAddEditObservationBinding.inflate(layoutInflater)
    setContentView(binding.root)

    setSupportActionBar(binding.toolbar)
    supportActionBar?.setDisplayHomeAsUpEnabled(true)

    viewModel =
ViewModelProvider(this).get(HikeViewModel::class.java)

    hikeId = intent.getLongExtra("HIKE_ID", -1)
    observationId = intent.getLongExtra("OBSERVATION_ID", -1)

    isEditMode = observationId != -1L

    if (hikeId == -1L && !isEditMode) {
        Toast.makeText(this, "Error: Invalid IDs",
Toast.LENGTH_SHORT).show()
        finish()
    }
}

```

```

        return
    }

    if (isEditMode) {
        supportActionBar?.title = "Edit Observation"
        binding.formContent.buttonSaveObservation.text = "Update
Observation"

        currentObservation =
intent.getParcelableExtra("OBSERVATION_OBJECT")
        if (currentObservation == null) {
            Toast.makeText(this, "Error loading observation",
Toast.LENGTH_SHORT).show()
            finish()
            return
        }
        hikeId = currentObservation!!.hikeId
        populateForm(currentObservation!!)

    } else {
        supportActionBar?.title = "Add Observation"
        updateTimeEditText()
    }

    setupDateTimePicker()
    setupButtons()
}

override fun onSupportNavigateUp(): Boolean {
    finish()
    return true
}

private fun populateForm(obs: Observation) {

binding.formContent.editTextObservation.setText(obs.observation)
    binding.formContent.editTextComments.setText(obs.comments)

    if (!obs.photoUri.isNullOrEmpty()) {

```

```

        currentPhotoUriString = obs.photoUri

binding.formContent.imageViewObservation.load(Uri.parse(currentPhotoU
riString)) {
    error(android.R.drawable.ic_menu_gallery)
}

    }

    val format = SimpleDateFormat("dd/MM/yyyy, HH:mm",
Locale.getDefault())
    try {
        calendar.time = format.parse(obs.time)!!
    } catch (e: Exception) {
    }
    updateTimeEditText()
}

private fun setupDateTimePicker() {
    binding.formContent.editTextTime.setOnClickListener {
        showDatePicker()
    }
}

private fun setupButtons() {
    binding.formContent.buttonSaveObservation.setOnClickListener {
        saveObservation()
    }
    binding.formContent.buttonTakePhoto.setOnClickListener {
        checkCameraPermissionAndLaunch()
    }
}

private fun showDatePicker() {
    val dateListener = DatePickerDialog.OnDateSetListener { _,
year, month, day ->
        calendar.set(Calendar.YEAR, year)
        calendar.set(Calendar.MONTH, month)
        calendar.set(Calendar.DAY_OF_MONTH, day)
        showTimePicker()
    }
}

```

```

    }
    DatePickerDialog(
        this, dateListener,
        calendar.get(Calendar.YEAR),
        calendar.get(Calendar.MONTH),
        calendar.get(Calendar.DAY_OF_MONTH)
    ).show()
}

private fun showTimePicker() {
    val timeListener = TimePickerDialog.OnTimeSetListener { _,
hour, minute ->
        calendar.set(Calendar.HOUR_OF_DAY, hour)
        calendar.set(Calendar.MINUTE, minute)
        updateTimeEditText()
    }
    TimePickerDialog(
        this, timeListener,
        calendar.get(Calendar.HOUR_OF_DAY),
        calendar.get(Calendar.MINUTE),
        true
    ).show()
}

private fun updateTimeEditText() {
    val format = SimpleDateFormat("dd/MM/yyyy, HH:mm",
Locale.getDefault())

binding.formContent.editTextTime.setText(format.format(calendar.time)
)
}

private fun saveObservation() {
    val observationText =
binding.formContent.editTextObservation.text.toString()
    val timeText =
binding.formContent.editTextTime.text.toString()
    val commentsText =
binding.formContent.editTextComments.text.toString()

```

```

        if (observationText.isEmpty() || timeText.isEmpty()) {
            Toast.makeText(this, "Observation and Time are required",
                Toast.LENGTH_SHORT).show()
            return
        }

        if (isEditMode) {
            val updatedObservation = Observation(
                id = currentObservation!!.id,
                hikeId = currentObservation!!.hikeId,
                observation = observationText,
                time = timeText,
                comments = commentsText,
                photoUri = currentPhotoUriString
            )
            viewModel.updateObservation(updatedObservation)
            Toast.makeText(this, "Observation updated",
                Toast.LENGTH_SHORT).show()
        } else {
            val observation = Observation(
                hikeId = hikeId,
                observation = observationText,
                time = timeText,
                comments = commentsText,
                photoUri = currentPhotoUriString
            )
            viewModel.addObservation(observation)
            Toast.makeText(this, "Observation saved",
                Toast.LENGTH_SHORT).show()
        }

        finish()
    }

    private fun getNewImageUri(): Uri {
        val imagePath = File(cacheDir, "images")
        imagePath.mkdirs()
    }

```

```

        val file = File(imagePath,
"obs_photo_${System.currentTimeMillis()}.jpg")
        return FileProvider.getUriForFile(this,
"${packageName}.provider", file)
    }

    private fun launchCamera() {
        val newUri = getNewImageUri()

        if (newUri != null) {
            photoUri = newUri
            takePictureLauncher.launch(newUri)
        } else {
            Toast.makeText(this, "Error creating image file",
Toast.LENGTH_SHORT).show()
        }
    }

    private fun checkCameraPermissionAndLaunch() {
        when {
            ContextCompat.checkSelfPermission(
                this,
                Manifest.permission.CAMERA
            ) == PackageManager.PERMISSION_GRANTED -> {
                launchCamera()
            }
        }

        shouldShowRequestPermissionRationale(Manifest.permission.CAMERA) -> {
            AlertDialog.Builder(this)
                .setTitle("Camera Permission")
                .setMessage("This app needs permission to take
photos for your observations.")
                .setPositiveButton("OK") { _, _ ->

cameraPermissionLauncher.launch(Manifest.permission.CAMERA)
                }
                .setNegativeButton("Cancel", null)
                .show()
        }
    }

```

```

        else -> {

cameraPermissionLauncher.launch(Manifest.permission.CAMERA)
        }
    }
}
}

```

[AdvancedSearchActivity.kt](#)

```

class AdvancedSearchActivity : AppCompatActivity() {

    private lateinit var binding: ActivityAdvancedSearchBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding =
ActivityAdvancedSearchBinding.inflate(layoutInflater)
        setContentView(binding.root)

        setSupportActionBar(binding.toolbar)
        supportActionBar?.setDisplayHomeAsUpEnabled(true)

        // Date Picker Logic
        binding.searchDate.setOnClickListener {
            val calendar = Calendar.getInstance()
            DatePickerDialog(this, { _, year, month, day ->
                val date = "$day/${month + 1}/${year}"
                binding.searchDate.setText(date)
            }, calendar.get(Calendar.YEAR),
calendar.get(Calendar.MONTH),
calendar.get(Calendar.DAY_OF_MONTH)).show()
        }

        // Search Button Logic
        binding.buttonPerformSearch.setOnClickListener {
            val name = binding.searchName.text.toString().trim()
            val location =

```

```

binding.searchLocation.text.toString().trim()
    val date = binding.searchDate.text.toString().trim()
    val length = binding.searchLength.text.toString().trim()

    // Start the Results Activity and pass the criteria
    val intent = Intent(this,
SearchResultsActivity::class.java)
    intent.putExtra("SEARCH_NAME", name)
    intent.putExtra("SEARCH_LOCATION", location)
    intent.putExtra("SEARCH_DATE", date)
    intent.putExtra("SEARCH_LENGTH", length)
    startActivity(intent)
}
}

override fun onSupportNavigateUp(): Boolean {
    finish()
    return true
}
}

```

[SearchResultsActivity.kt](#)

```

class SearchResultsActivity : AppCompatActivity() {

    private lateinit var binding: ActivitySearchResultsBinding
    private lateinit var viewModel: HikeViewModel
    private lateinit var hikeAdapter: HikeAdapter

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivitySearchResultsBinding.inflate(layoutInflater)
        setContentView(binding.root)

        setSupportActionBar(binding.toolbar)
        supportActionBar?.setDisplayHomeAsUpEnabled(true)

        viewModel =

```

```

ViewModelProvider(this).get(HikeViewModel::class.java)

    val name = intent.getStringExtra("SEARCH_NAME") ?: ""
    val location = intent.getStringExtra("SEARCH_LOCATION") ?: ""
    val date = intent.getStringExtra("SEARCH_DATE") ?: ""
    val length = intent.getStringExtra("SEARCH_LENGTH") ?: ""

    hikeAdapter = HikeAdapter(
        onClicked = { hike ->

            val intent = Intent(this,
HikeDetailActivity::class.java)
            intent.putExtra("HIKE_ID", hike.id)
            startActivity(intent)
        },
        onEditClicked = { hike ->
            val intent = Intent(this,
HikeDetailActivity::class.java)
            intent.putExtra("HIKE_ID", hike.id)
            startActivity(intent)
        }
    )
    binding.recyclerViewResults.adapter = hikeAdapter

    viewModel.advancedSearchHikes(name, location, length,
date).observe(this, Observer { hikes ->
        if (hikes.isEmpty()) {
            binding.textNoResults.visibility = View.VISIBLE
            binding.recyclerViewResults.visibility = View.GONE
        } else {
            binding.textNoResults.visibility = View.GONE
            binding.recyclerViewResults.visibility = View.VISIBLE
            hikeAdapter.submitList(hikes)
        }
    })
}

```

```

    override fun onSupportNavigateUp(): Boolean {
        finish()
        return true
    }
}

```

[HikeAdapter.kt](#)

```

class HikeAdapter(
    private val onCardClicked: (Hike) -> Unit,
    private val onEditClicked: (Hike) -> Unit
) :
    ListAdapter<Hike, HikeAdapter.HikeViewHolder>(HikeDiffCallback) {

    class HikeViewHolder(
        val binding: ListItemHikeBinding,
        private val onCardClicked: (Hike) -> Unit,
        private val onEditClicked: (Hike) -> Unit
    ) : RecyclerView.ViewHolder(binding.root) {

        fun bind(hike: Hike) {
            binding.textViewName.text = hike.name
            binding.textViewLocation.text = hike.location
            binding.textViewDate.text = hike.date
        }
    }

    // Called when RecyclerView needs a new row
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
        HikeViewHolder {
        val binding = ListItemHikeBinding.inflate(
            LayoutInflater.from(parent.context),
            parent,
            false
        )
    }
}

```

```

        return HikeViewHolder(binding, onCardClicked, onEditClicked)
    }

    // Called when RecyclerView wants to display data in a row
    override fun onBindViewHolder(holder: HikeViewHolder, position:
Int) {
        val currentHike = getItem(position)
        holder.bind(currentHike)

        holder.itemView.setOnClickListener {
            onCardClicked(currentHike)
        }
        holder.binding.buttonEditHike.setOnClickListener {
            onEditClicked(currentHike)
        }
    }
}

companion object HikeDiffCallback : DiffUtil.ItemCallback<Hike>()
{
    override fun areItemsTheSame(oldItem: Hike, newItem: Hike):
Boolean {
        return oldItem.id == newItem.id
    }

    override fun areContentsTheSame(oldItem: Hike, newItem: Hike):
Boolean {
        return oldItem == newItem
    }
}
}

```

[ObservationAdapter.kt](#)

```

class ObservationAdapter(
    private val onCardClicked: (Observation) -> Unit,
    private val onEditClicked: (Observation) -> Unit,

```

```

        private val onDeleteClicked: (Observation) -> Unit
    ) :
        ListAdapter<Observation,
        ObservationAdapter.ObservationViewHolder>(ObservationDiffCallback) {

        class ObservationViewHolder(
            val binding: ListItemObservationBinding,
            private val onCardClicked: (Observation) -> Unit,
            private val onEditClicked: (Observation) -> Unit,
            private val onDeleteClicked: (Observation) -> Unit
        ) :
            RecyclerView.ViewHolder(binding.root) {

            private var currentObservation: Observation? = null

            init {

                binding.root.setOnClickListener {
                    currentObservation?.let {
                        onCardClicked(it)
                    }
                }

                binding.buttonEditObservation.setOnClickListener {
                    currentObservation?.let {
                        onEditClicked(it)
                    }
                }

                binding.buttonDeleteObservation.setOnClickListener {
                    currentObservation?.let {
                        onDeleteClicked(it)
                    }
                }
            }

            fun bind(observation: Observation) {
                currentObservation = observation
                binding.textViewObservation.text = observation.observation
            }
        }
    }

```

```

        binding.textViewTime.text = observation.time

        // Load and show photo if it exists
        if (!observation.photoUri.isNullOrEmpty()) {
            binding.imageViewPhoto.visibility = View.VISIBLE

binding.imageViewPhoto.load(Uri.parse(observation.photoUri)) {

                placeholder(R.drawable.ic_launcher_background)
                error(android.R.drawable.ic_menu_gallery)
            }
        } else {

            binding.imageViewPhoto.visibility = View.GONE
        }

        if (observation.comments.isNullOrEmpty()) {
            binding.textViewComments.visibility = View.GONE
        } else {
            binding.textViewComments.text = observation.comments
        }
    }
}

override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
ObservationViewHolder {
    val binding = ListItemObservationBinding.inflate(
        LayoutInflater.from(parent.context),
        parent,
        false
    )
    return ObservationViewHolder(binding, onCardClicked,
onEditClicked, onDeleteClicked)
}

override fun onBindViewHolder(holder: ObservationViewHolder,
position: Int) {

```

```

        val observation = getItem(position)
        holder.bind(observation)

        val commentsView = holder.binding.textViewComments
        if (!observation.comments.isNullOrEmpty()) {

            holder.itemView.setOnClickListener {
                commentsView.visibility = if (commentsView.visibility
                == View.VISIBLE) View.GONE else View.VISIBLE

                onCardClicked(observation)
            }
        } else {
            holder.itemView.setOnClickListener {

                onCardClicked(observation)
            }
        }

        holder.binding.buttonEditObservation.setOnClickListener {
            onEditClicked(observation)
        }
        holder.binding.buttonDeleteObservation.setOnClickListener {
            onDeleteClicked(observation)
        }
    }

    companion object ObservationDiffCallback :
    DiffUtil.ItemCallback<Observation>() {
        override fun areItemsTheSame(oldItem: Observation, newItem:
        Observation): Boolean {
            return oldItem.id == newItem.id
        }

        override fun areContentsTheSame(oldItem: Observation, newItem:
        Observation): Boolean {
            return oldItem == newItem
        }
    }

```

```
}  
}
```

[add.js](#)

```
import DateTimePicker from "@react-native-community/datetimepicker";  
import { Picker } from "@react-native-picker/picker";  
import { useRouter } from "expo-router";  
import { useState } from "react";  
import {  
  Alert,  
  Button,  
  KeyboardAvoidingView,  
  Modal,  
  Platform,  
  SafeAreaView,  
  ScrollView,  
  StyleSheet,  
  Switch,  
  Text,  
  TextInput,  
  TouchableOpacity,  
  View,  
} from "react-native";  
  
import { addHike } from "../../lib/database";  
  
export default function AddHikeScreen() {  
  const router = useRouter();  
  
  const [name, setName] = useState("");  
  const [location, setLocation] = useState("");  
  const [date, setDate] = useState(new Date());  
  const [showDatePicker, setShowDatePicker] = useState(false);  
  const [parkingAvailable, setParkingAvailable] = useState(false);  
  const [length, setLength] = useState("");  
  const [description, setDescription] = useState("");  
  const [recommendedGear, setRecommendedGear] = useState("");
```

```

const [difficulty, setDifficulty] = useState("Easy");
const [showDifficultyPicker, setShowDifficultyPicker] =
useState(false);
const [trailCondition, setTrailCondition] = useState("Good");
const [showTrailPicker, setShowTrailPicker] = useState(false);

const handleSaveHike = async () => {
  console.log("hello :>> ");
  if (!name.trim() || !location.trim() || !length.trim()) {
    Alert.alert("Error", "Please fill in all required fields.");
    return;
  }
  if (isNaN(parseFloat(length))) {
    Alert.alert("Error", "Length must be a number (e.g., 5.5).");
    return;
  }
  Alert.alert(
    "Confirm Details",
    `Name: ${name}\nLocation: ${location}\nDate:
    ${date.toLocaleDateString()}\nParking: ${
      parkingAvailable ? "Yes" : "No"
    }\nLength: ${length} km\nDifficulty: ${difficulty}`,
    [
      { text: "Cancel", style: "cancel" },
      {
        text: "Confirm & Save",
        onPress: async () => {
          try {
            const hikeData = {
              name,
              location,
              date: date.toISOString().split("T")[0],
              parkingAvailable: parkingAvailable ? "Yes" : "No",
              length: parseFloat(length),
              difficulty,
              description,
              trailCondition,
              recommendedGear,
            };

```

```

        await addHike(hikeData);
        // Clear form after saving
        setName("");
        setLocation("");
        setDate(new Date());
        setParkingAvailable(false);
        setLength("");
        setDescription("");
        setRecommendedGear("");
        setDifficulty("Easy");
        setTrailCondition("Good");

        router.push("/");
    } catch (error) {
        console.error(error);
        Alert.alert("Error", "Failed to save hike.");
    }
  },
},
],
);
};

const onChange = (event, selectedDate) => {
  const currentDate = selectedDate || date;
  setShowDatePicker(Platform.OS === "ios");
  setDate(currentDate);
};

const toggleDatePicker = () => {
  setShowDatePicker(!showDatePicker);
};

return (
  <KeyboardAvoidingView
    style={styles.keyboardAvoidingContainer}
    behavior={Platform.OS === "ios" ? "padding" : "height"}
  >
    <ScrollView
      style={styles.container}

```

```

contentContainerStyle={styles.scrollContentContainer}
keyboardShouldPersistTaps="handled"
>
{/* (All your form inputs are correct) */}
<View style={styles.inputGroup}>
  <Text style={styles.label}>Name of Hike (Required)</Text>
  <TextInput
    style={styles.input}
    value={name}
    onChangeText={setName}
    placeholder="e.g., Snowdon"
    placeholderTextColor="#999"
  />
</View>

<View style={styles.inputGroup}>
  <Text style={styles.label}>Location (Required)</Text>
  <TextInput
    style={styles.input}
    value={location}
    onChangeText={setLocation}
    placeholder="e.g., Snowdonia, Wales"
    placeholderTextColor="#999"
  />
</View>

<View style={styles.inputGroup}>
  <Text style={styles.label}>Date of Hike (Required)</Text>
  {Platform.OS === "android" && (
    <TouchableOpacity
      onPress={toggleDatePicker}
      style={styles.dateButton}
    >
      <Text style={styles.dateButtonText}>
        {date.toLocaleDateString()}
      </Text>
    </TouchableOpacity>
  )}
  {(showDatePicker || Platform.OS === "ios") && (

```

```

        <DateTimePicker
            value={date}
            mode="date"
            display="default"
            onChange={onDateChange}
            style={styles.datePicker}
        />
    )}
</View>

<View style={styles.switchContainer}>
    <Text style={styles.label}>Parking Available?</Text>
    <Switch
        value={parkingAvailable}
        onValueChange={setParkingAvailable}
        trackColor={{ false: "#767577", true: "#2A9D8F" }}
        thumbColor={parkingAvailable ? "#f4f3f4" : "#f4f3f4"}
    />
</View>

<View style={styles.inputGroup}>
    <Text style={styles.label}>Length of Hike in km
(Required)</Text>
    <TextInput
        style={styles.input}
        value={length}
        onChangeText={setLength}
        placeholder="e.g., 14.5"
        keyboardType="numeric"
        placeholderTextColor="#999"
    />
</View>

<View style={styles.inputGroup}>
    <Text style={styles.label}>Difficulty (Required)</Text>
    <TouchableOpacity
        style={styles.pickerButton}
        onPress={() => setShowDifficultyPicker(true)}
    >

```

```

        <Text style={styles.pickerButtonText}>{difficulty}</Text>
      </TouchableOpacity>
    </View>

    <View style={styles.inputGroup}>
      <Text style={styles.label}>Description (Optional)</Text>
      <TextInput
        style={[styles.input, styles.multilineInput]}
        value={description}
        onChangeText={setDescription}
        placeholder="e.g., Beautiful views, very muddy..."
        multiline
        numberOfLines={4}
        placeholderTextColor="#999"
      />
    </View>

    <View style={styles.inputGroup}>
      <Text style={styles.label}>Trail Condition (Optional)</Text>
      <TouchableOpacity
        style={styles.pickerButton}
        onPress={() => setShowTrailPicker(true)}
      >
        <Text
style={styles.pickerButtonText}>{trailCondition}</Text>
      </TouchableOpacity>
    </View>

    {/* --- THIS IS THE FIX --- */}
    <View style={styles.inputGroup}>
      <Text style={styles.label}>Recommended Gear
(Optional)</Text>
      <TextInput
        style={styles.input}
        value={recommendedGear}
        onChangeText={setRecommendedGear}
        placeholder="e.g., Waterproof boots, extra water"
        placeholderTextColor="#999"
      />
    </View>

```

```

    </View>

    <TouchableOpacity style={styles.saveButton}
onPress={handleSaveHike}>
      <Text style={styles.saveButtonText}>Save Hike</Text>
    </TouchableOpacity>

    <View style={{ height: 50 }} />
  </ScrollView>

  {/* (All your Modal components are correct) */}
  <Modal
    transparent={true}
    visible={showDifficultyPicker}
    animationType="slide"
    onRequestClose={() => setShowDifficultyPicker(false)}
  >
    <SafeAreaView style={styles.modalContainer}>
      <View style={styles.modalContent}>
        <View style={styles.modalHeader}>
          <Button
            title="Done"
            onPress={() => setShowDifficultyPicker(false)}
          />
        </View>
        <Picker
          selectedValue={difficulty}
          onValueChange={(itemValue) => setDifficulty(itemValue)}
          style={styles.picker}
        >
          <Picker.Item label="Easy" value="Easy" />
          <Picker.Item label="Medium" value="Medium" />
          <Picker.Item label="Hard" value="Hard" />
        </Picker>
      </View>
    </SafeAreaView>
  </Modal>

  <Modal

```

```

        transparent={true}
        visible={showTrailPicker}
        animationType="slide"
        onRequestClose={() => setShowTrailPicker(false)}
    >
        <SafeAreaView style={styles.modalContainer}>
            <View style={styles.modalContent}>
                <View style={styles.modalHeader}>
                    <Button title="Done" onPress={() =>
setShowTrailPicker(false)} />
                </View>
                <Picker
                    selectedValue={trailCondition}
                    onChange={(itemValue) =>
setTrailCondition(itemValue)}
                    style={styles.picker}
                >
                    <Picker.Item label="Good" value="Good" />
                    <Picker.Item label="Fair" value="Fair" />
                    <Picker.Item label="Poor" value="Poor" />
                </Picker>
            </View>
        </SafeAreaView>
    </Modal>
</KeyboardAvoidingView>
);
}

// --- UPDATED Stylesheet ---
const styles = StyleSheet.create({
    // 5. Added styles for KeyboardAvoidingView
    keyboardAvoidingContainer: {
        flex: 1,
        backgroundColor: "#F4F7F6",
    },
    container: {
        flex: 1,
    },
    scrollContentContainer: {

```

```
padding: 20,
},
// (All other styles are correct)
inputGroup: {
  marginBottom: 18,
},
label: {
  fontSize: 16,
  fontWeight: "600",
  color: "#555",
  marginBottom: 8,
},
input: {
  backgroundColor: "#FFFFFF",
  borderWidth: 1,
  borderColor: "#DDDDDD",
  borderRadius: 8,
  paddingHorizontal: 15,
  paddingVertical: 12,
  fontSize: 16,
  color: "#333",
},
multilineInput: {
  height: 120,
  textAlignVertical: "top",
},
picker: {
  height: 215,
  width: "100%",
  color: "#333",
},
switchContainer: {
  flexDirection: "row",
  alignItems: "center",
  justifyContent: "space-between",
  backgroundColor: "#FFFFFF",
  borderWidth: 1,
  borderColor: "#DDDDDD",
  borderRadius: 8,
```

```
paddingHorizontal: 15,
paddingVertical: 10,
marginBottom: 18,
},
dateButton: {
  backgroundColor: "#FFFFFF",
  borderWidth: 1,
  borderColor: "#DDDDDD",
  borderRadius: 8,
  paddingHorizontal: 15,
  paddingVertical: 12,
},
dateButtonText: {
  fontSize: 16,
  color: "#333",
},
datePicker: {
  width: "100%",
  marginTop: 10,
},
saveButton: {
  backgroundColor: "#2A9D8F",
  paddingVertical: 15,
  borderRadius: 8,
  alignItems: "center",
  marginTop: 20,
},
saveButtonText: {
  color: "#FFFFFF",
  fontSize: 18,
  fontWeight: "bold",
},
pickerButton: {
  backgroundColor: "#FFFFFF",
  borderWidth: 1,
  borderColor: "#DDDDDD",
  borderRadius: 8,
  paddingHorizontal: 15,
  paddingVertical: 12,
```

```

    },
    pickerButtonText: {
      fontSize: 16,
      color: "#333",
    },
    modalContainer: {
      flex: 1,
      justifyContent: "flex-end",
      backgroundColor: "rgba(0, 0, 0, 0.5)",
    },
    modalContent: {
      backgroundColor: "#FFFFFF",
      borderTopLeftRadius: 20,
      borderTopRightRadius: 20,
    },
    modalHeader: {
      alignItems: "flex-end",
      padding: 10,
      borderBottomWidth: 1,
      borderBottomColor: "#eee",
    },
  },
});

```

[index.js](#)

```

import { Ionicons } from "@expo/vector-icons";
import { Link, Tabs, useFocusEffect } from "expo-router";
import React, { useState } from "react";
import {
  Alert,
  FlatList,
  SafeAreaView,
  StyleSheet,
  Text,
  TouchableOpacity,
  View,
} from "react-native";

```

```
import { deleteAllHikes, deleteHike, getAllHikes } from
"../../lib/database";

export default function HomeScreen() {
  const [hikes, setHikes] = useState([]);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(
    React.useCallback(() => {
      loadHikes();
    }, [])
  );

  const loadHikes = async () => {
    try {
      setIsLoading(true);
      const data = await getAllHikes();
      setHikes(data);
    } catch (error) {
      console.error(error);
      Alert.alert("Error", "Failed to load hikes.");
    } finally {
      setIsLoading(false);
    }
  };

  const handleResetDatabase = () => {
    Alert.alert(
      "Reset Database",
      "Are you sure you want to delete ALL hikes and observations?",
      [
        { text: "Cancel", style: "cancel" },
        {
          text: "DELETE",
          style: "destructive",
          onPress: async () => {
            await deleteAllHikes();
            loadHikes(); // Refresh the list
          },
        },
      ],
    );
  };
}
```

```

    },
  ]
);
};

const handleDeleteHike = (id) => {
  Alert.alert(
    "Delete Hike",
    "This will delete the hike and its observations. Continue?",
    [
      { text: "Cancel", style: "cancel" },
      {
        text: "DELETE",
        style: "destructive",
        onPress: async () => {
          try {
            setIsLoading(true);
            await deleteHike(id);
            await loadHikes();
          } catch (error) {
            console.error(error);
            setIsLoading(false);
            Alert.alert("Error", "Failed to delete hike.");
          }
        },
      },
    ],
  );
};

const renderHikeItem = ({ item }) => (
  <View style={styles.hikeItem}>
    <Link href={`/hike/${item.id}`} asChild>
      <TouchableOpacity style={styles.hikeInfo}>
        <View>
          <Text style={styles.hikeName}>{item.name}</Text>
          <Text style={styles.hikeLocation}>{item.location}</Text>
        </View>
        <Text style={styles.hikeDate}>

```

```

        {new Date(item.date).toLocaleDateString()}
      </Text>
    </TouchableOpacity>
  </Link>
  <TouchableOpacity
    accessibilityLabel={`Delete hike ${item.name}`}
    onPress={() => handleDeleteHike(item.id)}
    style={styles.deleteButton}
  >
    <Ionicons name="trash-outline" size={22} color="#E76F51" />
  </TouchableOpacity>
</View>
);

// This is shown if the list is empty
const renderEmptyListComponent = () => (
  <View style={styles.emptyContainer}>
    <Text style={styles.emptyText}>No Hikes Yet</Text>
    <Text style={styles.emptySubText}>Tap "Add New" to get
started!</Text>
  </View>
);

return (
  // SafeAreaView avoids the top notch and bottom bar
  <SafeAreaView style={styles.container}>
    {/* 3. This component sets the options for THIS tab screen */}
    <Tabs.Screen
      options={{
        headerRight: () => (
          <TouchableOpacity
            onPress={handleResetDatabase}
            style={{ marginRight: 15 }}
          >
            <Ionicons name="trash-outline" size={24} color="#E76F51"
/>
          </TouchableOpacity>
        ),
      }}
    />

```

```

    />

    <FlatList
      data={hikes}
      renderItem={renderHikeItem}
      keyExtractor={(item) => item.id.toString()}
      ListEmptyComponent={renderEmptyListComponent}
      contentContainerStyle={styles.listContainer}
      refreshing={isLoading}
      onRefresh={loadHikes} // Allows pull-to-refresh
    />
  </SafeAreaView>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "#F4F7F6", // Same light background as add-hike
  },
  listContainer: {
    flexGrow: 1, // Ensures empty list component can center
    padding: 15,
  },
  hikeItem: {
    backgroundColor: "#FFFFFF",
    padding: 20,
    borderRadius: 12,
    marginBottom: 15,
    flexDirection: "row",
    alignItems: "center",
    // --- Shadow for iOS ---
    shadowColor: "#000",
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.05,
    shadowRadius: 4,
    // --- Shadow for Android ---
    elevation: 3,
  },
},

```

```
hikeName: {
  fontSize: 18,
  fontWeight: "600", // Semibold
  color: "#333",
},
hikeLocation: {
  fontSize: 14,
  color: "#666",
  marginTop: 4,
},
hikeDate: {
  fontSize: 14,
  color: "#2A9D8F", // Use our theme green
  fontWeight: "500",
},
hikeInfo: {
  flex: 1,
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
},
deleteButton: {
  padding: 8,
  marginLeft: 12,
},
// --- Empty List Styles ---
emptyContainer: {
  flex: 1,
  justifyContent: "center",
  alignItems: "center",
  marginTop: -50, // Adjust vertical position
},
emptyText: {
  fontSize: 20,
  fontWeight: "bold",
  color: "#555",
},
emptySubText: {
  fontSize: 16,
```

```
    color: "#777",
    marginTop: 10,
    textAlign: "center",
    paddingHorizontal: 20,
  },
});
```

[search.js](#)

```
import { Link } from "expo-router";
import { useState } from "react";
import {
  Alert,
  FlatList,
  SafeAreaView,
  StyleSheet,
  Text,
  TextInput,
  TouchableOpacity,
  View,
} from "react-native";
import { searchHikesByName } from "../../lib/database";

export default function SearchScreen() {
  const [nameQuery, setNameQuery] = useState("");
  const [results, setResults] = useState([]);

  const handleSearch = async (query) => {
    setNameQuery(query);

    if (!query.trim()) {
      setResults([]);
      return;
    }

    try {
      const data = await searchHikesByName(query.trim());
```

```

    setResults(data);
  } catch (error) {
    console.error(error);
    Alert.alert("Error", "Search failed.");
  }
};

const renderHikeItem = ({ item }) => (
  <Link href={`/hike/${item.id}`} asChild>
    <TouchableOpacity style={styles.hikeItem}>
      <View>
        <Text style={styles.hikeName}>{item.name}</Text>
        <Text style={styles.hikeLocation}>{item.location}</Text>
      </View>
      <Text style={styles.hikeDate}>
        {new Date(item.date).toLocaleDateString()}
      </Text>
    </TouchableOpacity>
  </Link>
);

const renderEmpty = () => (
  <View style={styles.emptyContainer}>
    <Text style={styles.emptyText}>
      {nameQuery.length > 0
        ? "No results found"
        : "Start typing to search for a hike"}
    </Text>
  </View>
);

return (
  <SafeAreaView style={styles.container}>
    {/* --- CHANGE 1: Search bar is now outside the FlatList --- */}
    <View style={styles.searchSection}>
      <TextInput
        style={styles.input}
        value={nameQuery}
        onChangeText={handleSearch}

```

```

        placeholder="Enter hike name..."
        placeholderTextColor="#999"
        clearButtonMode="always"
    />
</View>

<FlatList
  data={results}
  renderItem={renderHikeItem}
  keyExtractor={(item) => item.id.toString()}
  ListEmptyComponent={renderEmpty}
  contentContainerStyle={styles.listContentContainer}
  keyboardShouldPersistTaps="handled"
/>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "#F4F7F6",
  },

  searchSection: {
    padding: 15,
    backgroundColor: "#F4F7F6",
  },

  listContentContainer: {
    paddingHorizontal: 15,
    paddingBottom: 15,
  },
  input: {
    backgroundColor: "#FFFFFF",
    borderWidth: 1,
    borderColor: "#DDDDDD",
    borderRadius: 8,
    paddingHorizontal: 15,

```

```
paddingVertical: 12,  
fontSize: 16,  
color: "#333",  
},  
  
hikeItem: {  
  backgroundColor: "#FFFFFF",  
  padding: 20,  
  borderRadius: 12,  
  marginBottom: 15,  
  flexDirection: "row",  
  justifyContent: "space-between",  
  alignItems: "center",  
  shadowColor: "#000",  
  shadowOffset: { width: 0, height: 2 },  
  shadowOpacity: 0.05,  
  shadowRadius: 4,  
  elevation: 3,  
},  
  
hikeName: {  
  fontSize: 18,  
  fontWeight: "600",  
  color: "#333",  
},  
  
hikeLocation: {  
  fontSize: 14,  
  color: "#666",  
  marginTop: 4,  
},  
  
hikeDate: {  
  fontSize: 14,  
  color: "#2A9D8F",  
  fontWeight: "500",  
},  
  
emptyContainer: {  
  paddingTop: 50,  
  alignItems: "center",  
},
```

```
emptyText: {
  fontSize: 16,
  color: "#777",
},
});
```

hike/add-ons/[id].js

```
import { useLocalSearchParams, useRouter } from "expo-router";
import { useState } from "react";
import {
  Alert,
  KeyboardAvoidingView,
  Platform,
  ScrollView,
  StyleSheet,
  Text,
  TextInput,
  TouchableOpacity,
  View,
} from "react-native";
import { addObservation } from "../../lib/database";

export default function AddObservationScreen() {
  const router = useRouter();
  const { id } = useLocalSearchParams();

  const [observation, setObservation] = useState("");
  const [comments, setComments] = useState("");

  const handleSave = async () => {
    if (!observation.trim()) {
      Alert.alert("Error", "Observation field is required.");
      return;
    }
  }

  try {
    await addObservation({
      hike_id: id,
      observation: observation,
      time: new Date().toISOString(),
      comments: comments,
    });
    // Go back to the previous screen
    router.back();
  } catch (error) {
```

```

    console.error(error);
    Alert.alert("Error", "Failed to add observation.");
  }
};

return (
  <KeyboardAvoidingView
    style={styles.keyboardAvoidingContainer}
    behavior={Platform.OS === "ios" ? "padding" : "height"}
  >
    <ScrollView
      style={styles.container}
      contentContainerStyle={styles.scrollContentContainer}
      keyboardShouldPersistTaps="handled"
    >
      <View style={styles.inputGroup}>
        <Text style={styles.label}>Observation (Required)</Text>
        <TextInput
          style={[styles.input, styles.multilineInput]}
          value={observation}
          onChangeText={setObservation}
          placeholder="e.g., Saw a deer, trail was muddy"
          placeholderTextColor="#999"
          multiline
          numberOfLines={4}
        />
      </View>

      <View style={styles.inputGroup}>
        <Text style={styles.label}>Additional Comments (Optional)</Text>
        <TextInput
          style={[styles.input, styles.multilineInput]}
          value={comments}
          onChangeText={setComments}
          placeholder="e.g., Near the old oak tree"
          placeholderTextColor="#999"
          multiline
          numberOfLines={2}
        />
      </View>

      <TouchableOpacity style={styles.saveButton} onPress={handleSave}>
        <Text style={styles.saveButtonText}>Add Observation</Text>
      </TouchableOpacity>
    </ScrollView>
  </KeyboardAvoidingView>
);
}

// --- Stylesheet (copied from add-hike.js for consistency) ---
const styles = StyleSheet.create({
  keyboardAvoidingContainer: {

```

```
    flex: 1,
    backgroundColor: "#F4F7F6",
  },
  container: {
    flex: 1,
  },
  scrollContentContainer: {
    padding: 20,
  },
  inputGroup: {
    marginBottom: 18,
  },
  label: {
    fontSize: 16,
    fontWeight: "600",
    color: "#555",
    marginBottom: 8,
  },
  input: {
    backgroundColor: "#FFFFFF",
    borderWidth: 1,
    borderColor: "#DDDDDD",
    borderRadius: 8,
    paddingHorizontal: 15,
    paddingVertical: 12,
    fontSize: 16,
    color: "#333",
  },
  multilineInput: {
    height: 120,
    textAlignVertical: "top",
  },
  saveButton: {
    backgroundColor: "#2A9D8F",
    paddingVertical: 15,
    borderRadius: 8,
    alignItems: "center",
    marginTop: 20,
  },
  saveButtonText: {
    color: "#FFFFFF",
    fontSize: 18,
    fontWeight: "bold",
  },
});
```